

Chest X-Ray Classification: Pneumonia and Normal

Project Done By Jackelyn Valencia Dataset obtained from:

<https://www.kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia/>

Technical Report

Description of Dataset

This dataset contains chest x-ray images, where some are classified as "pneumonia" and the rest as "normal". The dataset is split into two different sets: one called "Train" and the other called "Test". We will be using these sets to train and test some algorithms, such as PCA and Neural Networks, maybe kmeans, that we will use later on. The train set contains a total of 5216 images: 1341 normal and 3875 pneumonia. The test set contains a total of 624 images: 390 normal and 234 pneumonia.

Analysis (Checkpoint 2)

I analysed the train data by using PCA, which was also used for feature extraction. We wanted to see if we could see any difference between the pneumonia images and normal ones. I used PCA on 3 different sets of data: both normal and pneumonia, pneumonia only, normal only. The explained variance was

For the one containing both normal and pneumonia, I could not detect any difference between pneumonia and normal with my eyes.

For the one containing only pneumonia, The feature extraction worked a little. The images seemed more blurry, which could be due to the people whose image was taken had pneumonia. Therefore, if the image was blurry, it could be due to pneumonia and classified for it.

For the one containing only normal, I could see the feature extraction work. The bones in the chest xray seemed to be highlighted alot from here. Most likely since the image was more clearer due to those whose image was taken did not have pneumonia.

The explained variance for $n = 3$ components was [0.24935416 0.10293229 0.07142283]. Therefore the PCA was not able to bring out the image features well.

Classification and Clustering Algorithms Used

I used a Convolutional Neural Network to classify the images into their corresponding labels. I used this because the images can be difficult to determine what is a chest ray that is classified as 'pneumonia' or 'normal'. Because it can sometimes be difficult to label them even with the human eye, by using a neural network, we can find features that can help it determine what makes a chest ray image a 'pneumonia' or 'normal' image. It is also commonly used for images with a lot of noise, which our images have.

I also used Kmeans clustering, however, it wasn't doing as well as CNN since the dataset was highly unbalanced and I don't think I plotted the points well,

HyperParameters

There were several changes I did with my classification algorithm to try to increase the accuracy rate. At first, my accuracy was 36.3% for normal and 99.7% for pneumonia. This meant that the algorithm would most likely predict images to be pneumonia, which is why it had a higher rate. Out of all of the changes I did, adding the class weights in CrossEntropyLoss and greater bin size really helped increase it, mainly for the normal images to 53.8%.

Discussion

What I learned about my dataset

After analysing the dataset by viewing the images through the use of PCA and TSNE, I concluded that it can be difficult to classify images to its respective class. I have also tried classifying through my own eyes but I tended to classify all blurry images or highly concentrated areas with white noise to be pneumonia. However, there would be times where those images were not actually pneumonia but normal. So even I had trouble classifying them through my own eyes. I also noticed that since these are xray images, they would be similar in structure since our chest xrays look similar for those with pneumonia and without. Through PCA, we were able to visualize how the images would look like. From the Analysis on Checkpoint 2, we saw that the chest xrays were highlighted more for those classified as normal, and it was more blurry for those with pneumonia. Therefore, there was more noise on the pneumonia images than on the normal ones.

What I learned about using algorithms in my project

In my project, I had used PCA and TSNE for visualization. As well as Kmeans Clustering and Convolutional NN to cluster and classify the images. Since this project's main goal was to classify the images into their respective category, using a neural network would be best as it deals with more noisy images. Through the use of PCA and TSNE, we were able to see if we could separate the points and see a clear distinction between them. However, we could see that the points would be on top of each other on both PCA and TSNE, so it was not possible to see any distinctions between them. Therefore when both normal and pneumonia images are together, it can be difficult to determine which is which.

Through the use of Kmeans, I tried to see if I could cluster the images in some way to see if Kmeans could do a better job of visualizing these points. However, it also could not do so as the points again overlapped over each other. I would probably need more time to work with this algorithm as well since I did not work on it as much as I had with the other algorithms.

Through the use of Neural networks, I tried different changes to see if I could improve the accuracy. As stated before, my accuracy at the beginning without any changes was 36.3% for normal and 99.7% for pneumonia. Since the dataset was highly unbalanced for both train and test images, I created a Class Weight to reduce the prediction of images to be pneumonia. By adding this, I was able to increase the accuracy rate up to 53.0% for pneumonia. I also tried to

reduce the learning rate from 0.001 to 0.0001 as there was the idea that the neural network could be stuck in between each other if the learning rate was too big. However, this did not improve the algorithm by much. Next, we tried a different batch size from 4 to 32. This would allow us to view more images at a time and to speed up the process a little bit faster. This also increased the accuracy rate for normal class to 53.8%. I wanted to try to train my network for longer epochs but it was taking too long so the results were not reported. I tried using a different optimizer as well called adam to see if my results would change but the accuracy was not as good as the others. Overall, I believe that changing the class weights and increasing the batch size improved the classification of the images. I believe that training the network for a longer time and making more changes would help increase the prediction for normal classes better than before.

Contribution/Resources Used;

Used as reference on how to use the packages functions

- <https://scikit-learn.org/>
- <https://numpy.org/>
- <https://pandas.pydata.org/>

Used as reference for kmeans clustering

<https://www.uniovi.es/compnum/labs/PYTHON/kmeans.html>

<https://www.geeksforgeeks.org/k-means-clustering-introduction/>

Used as reference on how to use and create CNN

- <https://pytorch.org/>
- https://spotintelligence.com/2023/03/01/adam-optimizer/#2_PyTorch_Adam_optimizer
- <https://medium.com/bitgrit-data-science-publication/building-an-image-classification-model-with-pytorch-from-scratch-f10452073212>
- <https://www.quora.com/What-is-max-pooling-in-convolutional-neural-networks>
- <https://stats.stackexchange.com/questions/380996/convolutional-network-how-to-choose-output-channels-number-stride-and-padding/381032#381032>
- https://pytorch.org/vision/stable/auto_examples/transforms/plot_transforms_illustrations.html#sglr-auto-examples-transforms-plot-transforms-illustrations-py

Outline of Steps in Project

- Creating my Dataframe for PCA
 - View Images (To see what I'm working with)
 - Creating Train and Test Dataframes
- KMeans Clustering (need to work on it more)
- PCA on Train Dataset (Analysis from Part 2)
 - PCA on both normal and pneumonia

- PCA on pneumonia only
- PCA on normal only
- TSNE on Train Dataset
- Convolutional Neural Network for Image Classification
 - Change 1: Adding Class Weight
 - Change 2: Trying a Different Hyperparameter (learning rate = 0.0001)
 - Change 3: Trying a Different Batch Size (batch_size = 32)
 - Change 4: Training our Network for Longer epochs (epochs = 50)
 - Change 5: Trying a Different Optimizer (ADAM)

Creating my Dataframe for PCA

```
In [2]: # import libraries

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from matplotlib import pyplot as plt
import sys
import os
import cv2
# Go to conda activate myenv
# Go to Data Mining Folder
# Use jupyter-lab
import time
import seaborn as sns
from sklearn import cluster, decomposition
```

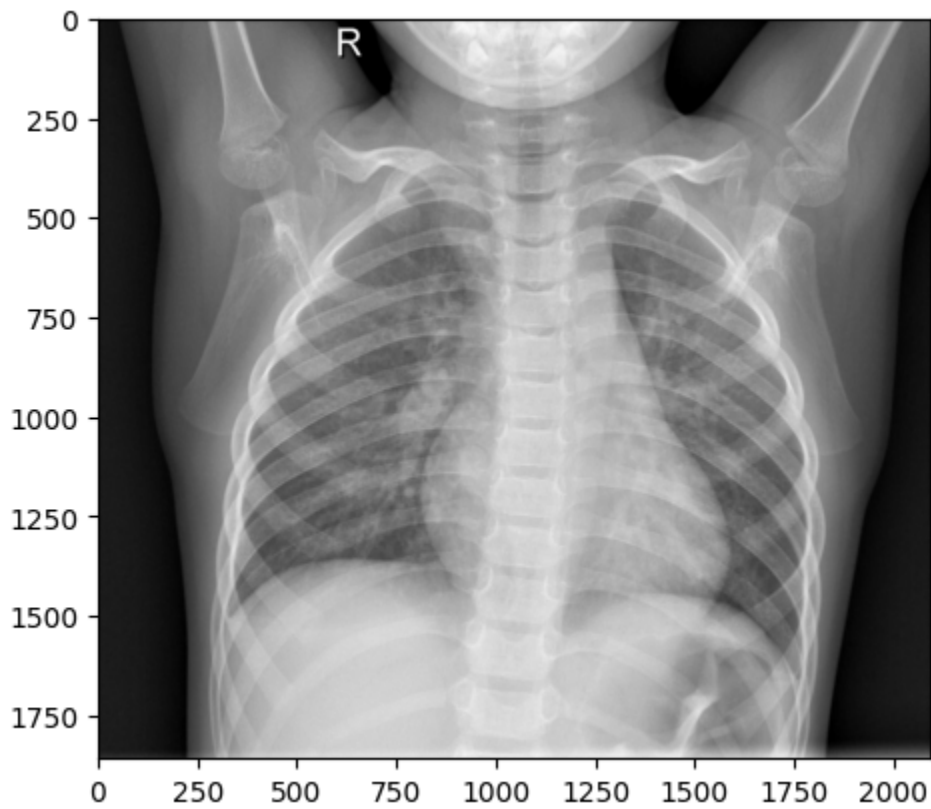
Viewing Images

```
In [3]: # using cv2 to view a 'normal' xray image
# Read Image
train_path = 'C:/Users/DellDepot/Downloads/chest_xray/train/NORMAL/IM-0115-000'
im = cv2.imread(train_path)

# Show Image
plt.imshow(im)

# The shape of this image is (1858, 2090, 3)
im.shape
```

```
Out[3]: (1858, 2090, 3)
```



```
In [4]: # using cv2 to view another image

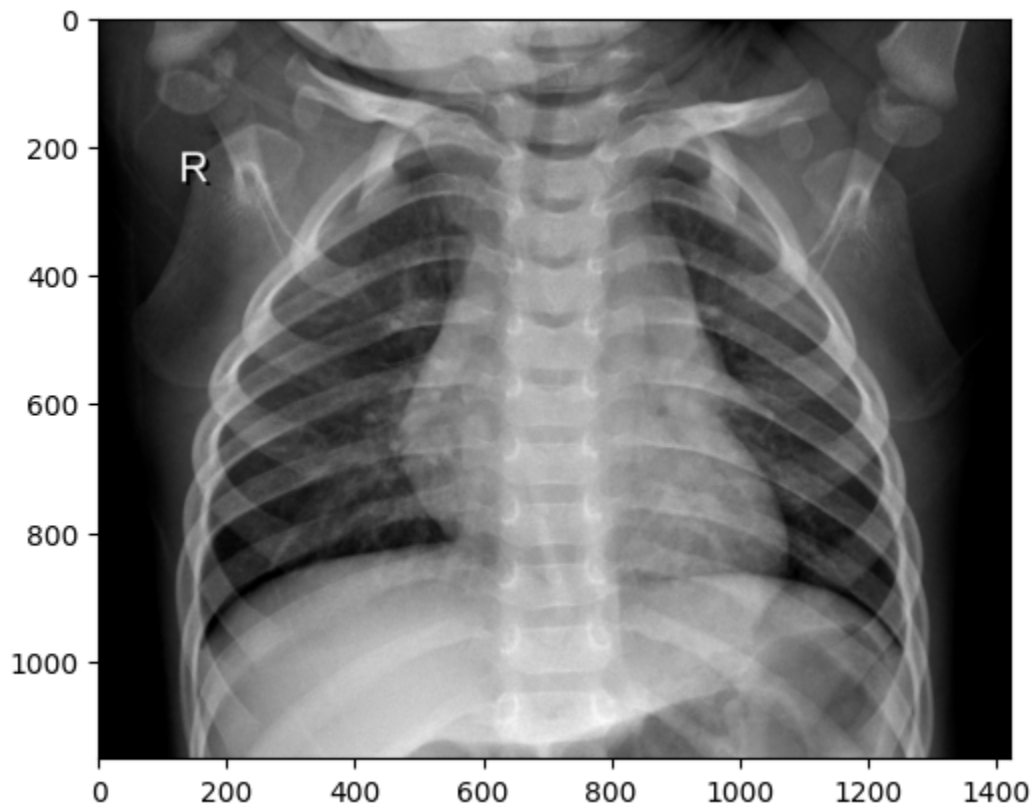
# This is a normal xray

# Read Image
train_path = 'C:/Users/DellDepot/Downloads/chest_xray/train/NORMAL/IM-0117-000
im = cv2.imread(train_path)

# Show Image
plt.imshow(im)

# The shape of this image is (1152, 1422, 3)
im.shape
```

Out[4]: (1152, 1422, 3)



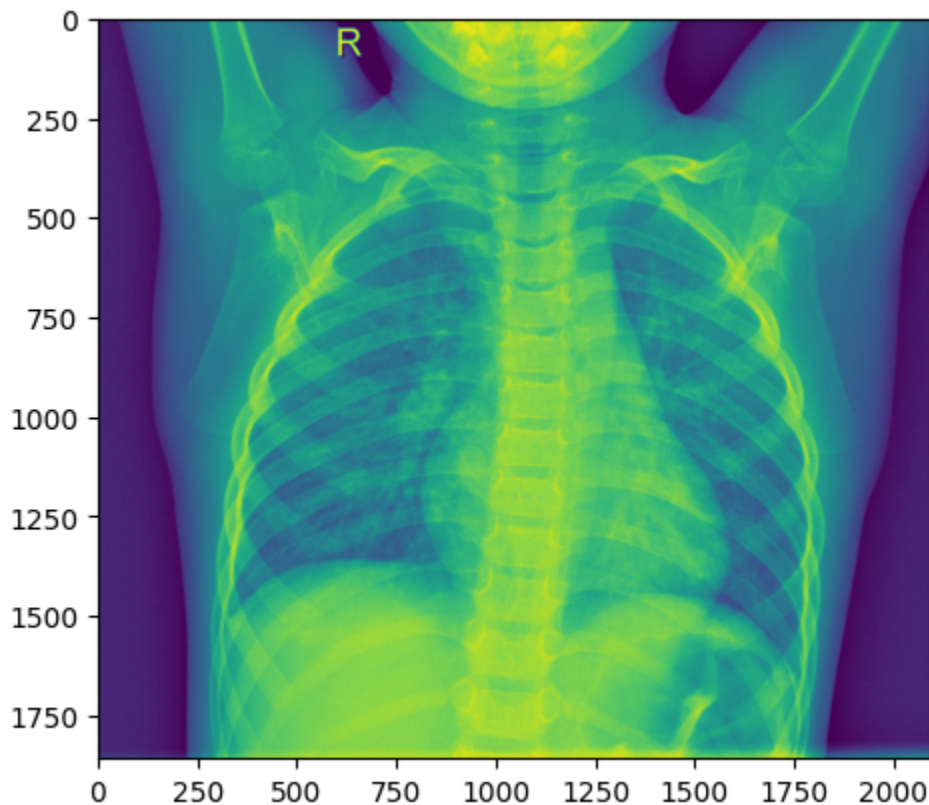
```
In [5]: # view first image using plt, normal xray
import matplotlib.image as mpimg
import matplotlib.pyplot as plt

# Read Images
img = mpimg.imread('C:/Users/DellDepot/Downloads/chest_xray/train/NORMAL/IM-01

# Output Images
plt.imshow(img)

# Same shape as before
print(img.shape)

(1858, 2090)
```



Since the images in the dataset had different sizes, I decided to grayscale my images and then resize them to (100,100), that way they all have the same size to be analyzed later

Creating Train and Test Dataframes

```
In [6]: # Since our images will be 100 x 100, we will flatten each image into an array
# So each row is an image that contains the 10000 pixels of the image
grayscale_cols = 10000
```

```
In [7]: # This function process each image by taking in the location of the image and
# grayscale image, and resizing it to a 100 by 100 image, and then flattening

# inputs:
# - image_path: location of the image you want to process
# - is_grayscale = default;False: if false, it will not turn image into grayscale
#           turn image into grayscale
# - img_size = default;100: turns image into 100*100, or you can set your own size
def process_img(image_path, is_grayscale=False, img_size=100):
    img = cv2.imread(image_path)
    if is_grayscale:
        img = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
    img = cv2.resize(img, (img_size, img_size))
    img = img.astype('float') / 255
    img = img.flatten()
    img = pd.Series(img)
    return img

# This function creates my dataframe and returns

# inputs:
# - cols: number of columns in the dataframe
# - type_xray: gets the xray type ("NORMAL", "PNEUMONIA") from folder, we
# - is_grayscale = default; False
```



```

# - img_size = default; 100
def process_image_set(cols, type_xray, is_grayscale=False, img_size=100):
    start = time.time()
    cols = np.arange(cols)
    df = pd.DataFrame(columns=cols)
    labelcol = []
    x = 0
    for subdir, _, files in os.walk(raw_data):
        for file in files:
            img_path = subdir + os.sep + file
            img = process_img(img_path, is_grayscale, img_size)
            df.loc[x] = img
            x += 1
            labelcol.append(type_xray)
    df['label'] = labelcol
    print("Dataset finished processing")
    end = time.time()
    print(f"Time elapsed: {end - start}")
    return df

```

In [157... *# Lets create our 'normal' grayscaled xray images dataframe*

```

# Get pathway to the train dataset which accesses the "NORMAL" chest x-rays
raw_data = 'C:/Users/DellDepot/Downloads/chest_xray/train/NORMAL'
grayscale_df_normal_train = process_image_set(grayscale_cols, "NORMAL", is_grayscale=True)

# Create our 'pneumonia' grayscaled xray images dataframe
raw_data = 'C:/Users/DellDepot/Downloads/chest_xray/train/PNEUMONIA'
grayscale_df_pneumonia_train = process_image_set(grayscale_cols, "PNEUMONIA", is_grayscale=True)

# Join dataframes together, they make up the train dataframe
frames = [grayscale_df_normal_train, grayscale_df_pneumonia_train]
grayscale_df_train = pd.concat(frames, ignore_index=True)

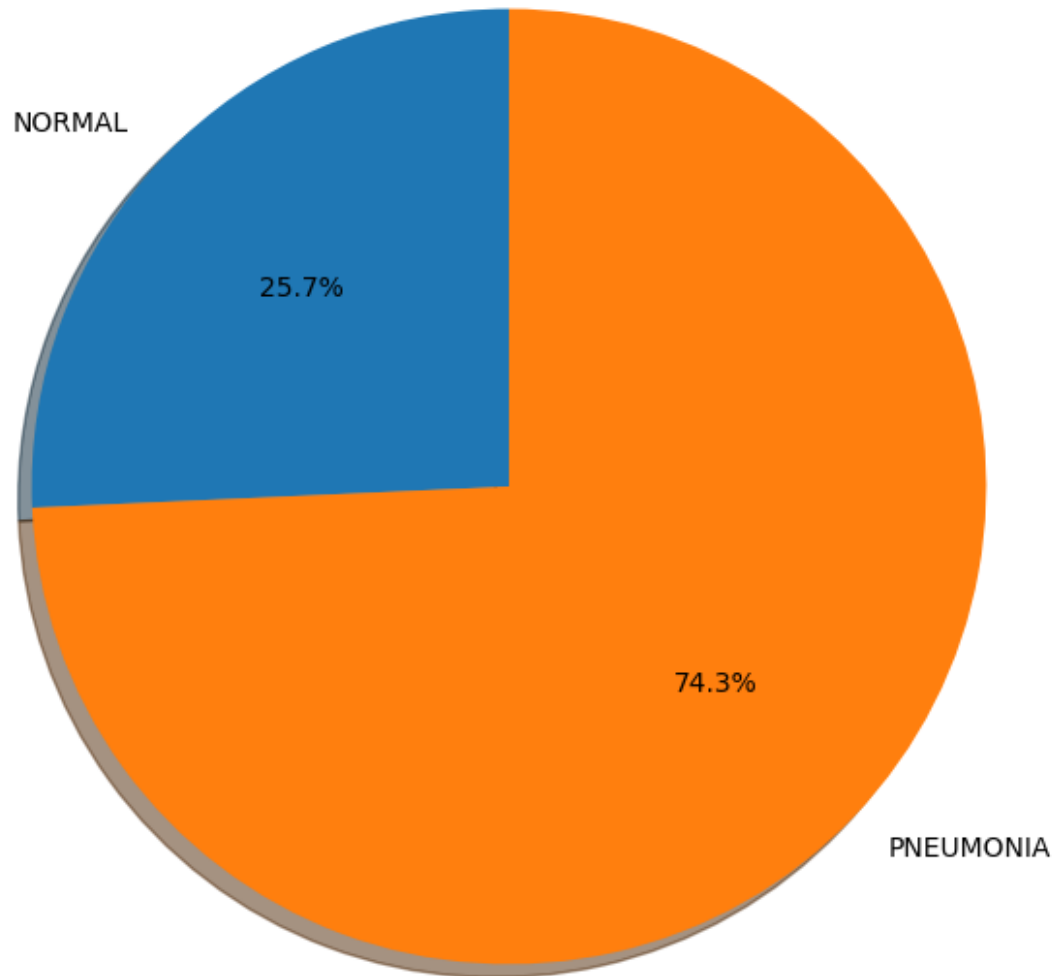
# We can visualize our train dataset here
# IMPORTANT: Shows that our train dataset is unbalanced
plt.figure(figsize = (8,8))
plt.pie(grayscale_df_train.groupby('label').size(), labels = ['NORMAL', 'PNEUMONIA'])
plt.show()

```

```

Dataset finished processing
Time elapsed: 114.35200023651123
Dataset finished processing
Time elapsed: 671.9361593723297

```

```
In [158... # We will create our grayscale test dataframe to visualize as well

# Create our 'normal' grayscaled xray images dataframe
raw_data = 'C:/Users/DellDepot/Downloads/chest_xray/test/NORMAL'
grayscale_df_normal_test = process_image_set(grayscale_cols, "NORMAL", is_gray

# Create our 'pneumonia' grayscaled xray images dataframe
raw_data = 'C:/Users/DellDepot/Downloads/chest_xray/test/PNEUMONIA'
grayscale_df_pneumonia_test = process_image_set(grayscale_cols, "PNEUMONIA", i

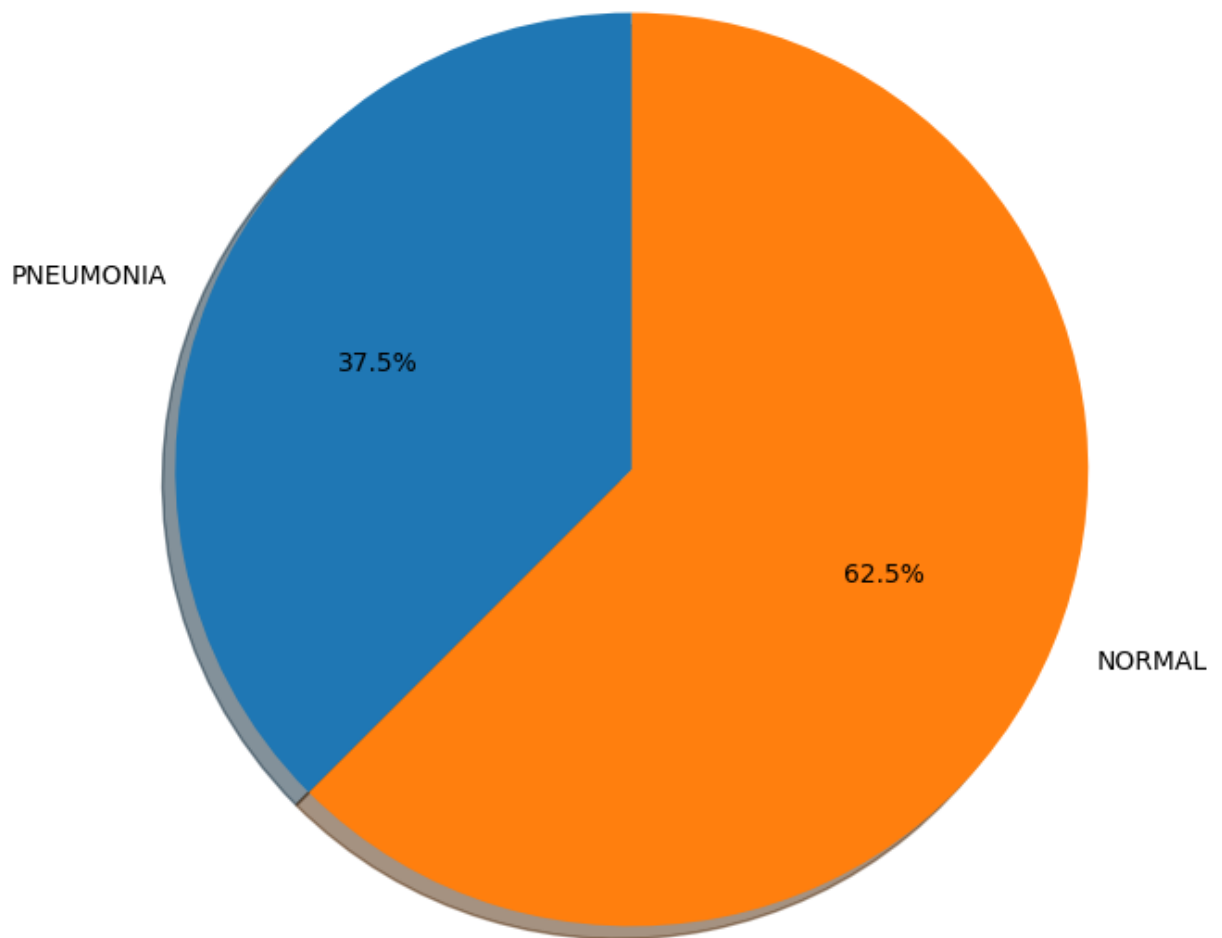
# Join dataframes together, they make up the test dataframe
frames = [grayscale_df_normal_test, grayscale_df_pneumonia_test]

grayscale_df_test = pd.concat(frames, ignore_index=True)

# We can visualize our test dataset here
# IMPORTANT: Shows that our train dataset is unbalanced
plt.figure(figsize = (8,8))
plt.pie(grayscale_df_test.groupby('label').size(), labels = ['PNEUMONIA', 'NORM
plt.show()
```

Dataset finished processing
Time elapsed: 12.701969146728516

Dataset finished processing
Time elapsed: 14.716272354125977



K Means Clustering

```
In [160... from sklearn.cluster import KMeans

data = grayscale_df_train_imageonly.to_numpy()
print(type(grayscale_df_train_imageonly))

n = 2
kmeans = KMeans(n_clusters = n, init = 'random')
kmeans.fit(data)
Z = kmeans.predict(data)

#print(Z)

for i in range(0,n):

    row = np.where(Z==i)[0] # row in Z for elements of cluster i
    #print("row",row)
    num = row.shape[0]      # number of elements for each cluster
    #print("number of shape",num)
```

```

r = int(np.floor(num/10.))    # number of rows in the figure of the cluster

print("cluster "+str(i))
print(str(num)+" elements")

plt.figure(figsize=(2,10))
for k in range(0, 20):
    print(r, k)

    plt.subplot(2, 10, k+1)
    #print(row[k])
    #print(data[row[k]])
    #print(row[k])
    image = data[row[k]]
    image = image.reshape((100, 100))#image = image.reshape(8, 8)
    plt.imshow(image, cmap='gray')
    plt.axis('off')
plt.show()

```

```
<class 'pandas.core.frame.DataFrame'>
```

```

C:\Users\DellDepot\anaconda3\envs\myenv\lib\site-packages\sklearn\cluster\_kmeans.py:1416: FutureWarning: The default value of `n_init` will change from 10 to 'auto' in 1.4. Set the value of `n_init` explicitly to suppress the warning
  super()._check_params_vs_input(X, default_n_init=10)

```

```

cluster 0
2579 elements
257 0
257 1
257 2
257 3
257 4
257 5
257 6
257 7
257 8
257 9
257 10
257 11
257 12
257 13
257 14
257 15
257 16
257 17
257 18
257 19

```

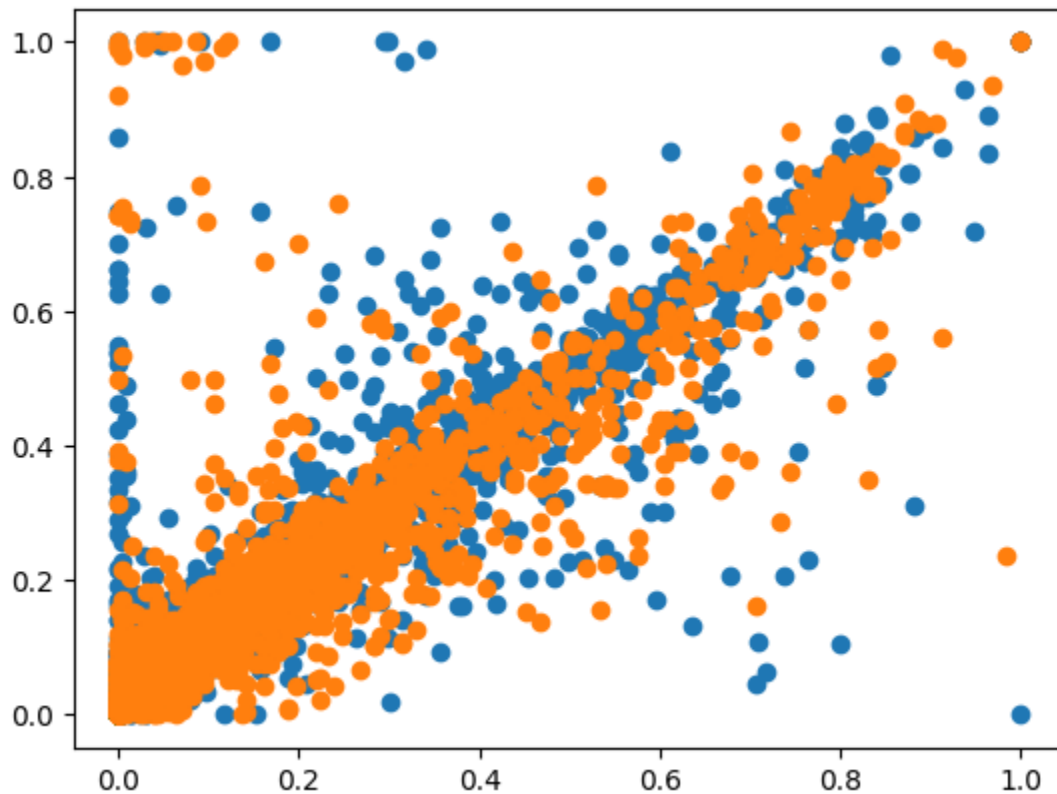


cluster 1
2637 elements
263 0
263 1
263 2
263 3
263 4
263 5
263 6
263 7
263 8
263 9
263 10
263 11
263 12
263 13
263 14
263 15
263 16
263 17
263 18
263 19



```
In [161... centroids = kmeans.cluster_centers_  
labels= kmeans.labels_
```

```
In [162... # Plot the results  
for i in set(kmeans.labels_):  
    index = kmeans.labels_ == i  
    plt.plot(data[index,0], data[index,1], 'o')  
plt.show()
```



```
In [108]: grayscale_df_train_imageonly.shape
```

```
Out[108]: (5216, 10000)
```

PCA Testing on Train Dataset

PCA on Both Pneumonia and Normal

```
In [105]: # drop label to see only the image pixels
grayscale_df_train_imageonly = grayscale_df_train.drop('label', axis=1)

# randomize dataset to view images later
grayscale_df_train_imageonly = grayscale_df_train_imageonly.sample(frac = 1).r
grayscale_df_train_imageonly = grayscale_df_train_imageonly.drop('index', axis=1)

# turn image points to numpy
grayscale_df_train_imageonly_array = grayscale_df_train_imageonly.to_numpy()

# Centering of data
faces_centered = grayscale_df_train_imageonly_array - grayscale_df_train_image
# Print faces_centered
faces_centered

# Normalize data
faces_centered -= faces_centered.mean(axis=1).reshape(5216, -1)
```

```
In [14]: # Used this from class
# looking at a few components

n_row, n_col = 5, 6
n_components = n_row * n_col
```

```

image_shape = (100, 100)

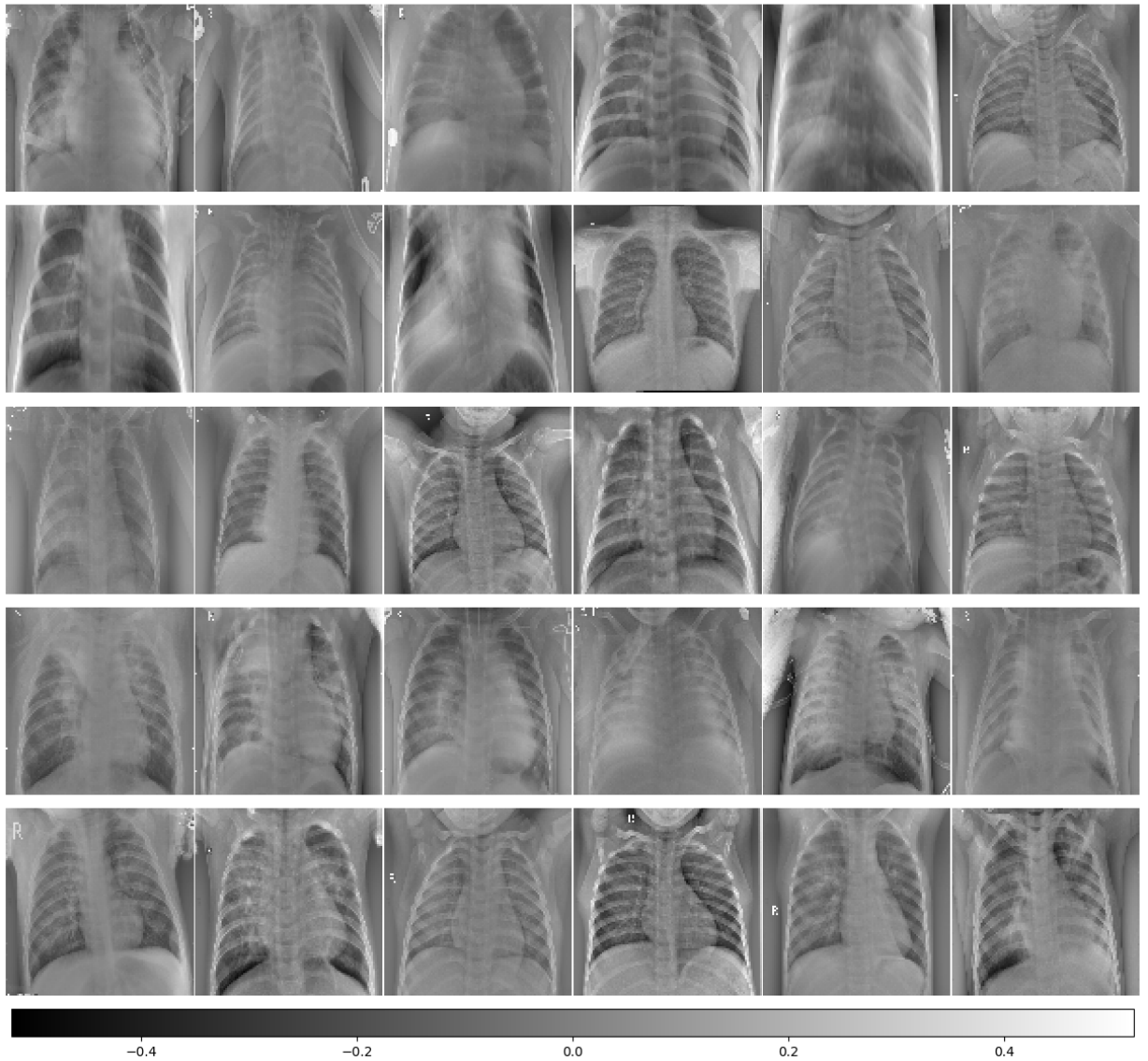
def plot_gallery(title, images, n_col=n_col, n_row=n_row, cmap=plt.cm.gray):
    fig, axs = plt.subplots(
        nrow=n_row,
        ncol=n_col,
        figsize=(2.0 * n_col, 2.3 * n_row),
        facecolor="white",
        constrained_layout=True,
    )
    fig.set_constrained_layout_pads(w_pad=0.01, h_pad=0.02, hspace=0, wspace=0)
    fig.set_edgecolor("black")
    fig.suptitle(title, size=16)
    for ax, vec in zip(axs.flat, images):
        vmax = max(vec.max(), -vec.min())
        im = ax.imshow(
            vec.reshape(image_shape),
            cmap=cmap,
            interpolation="nearest",
            vmin=-vmax,
            vmax=vmax,
        )
        ax.axis("off")

    fig.colorbar(im, ax=axs, orientation="horizontal", shrink=0.99, aspect=40,
plt.show()

```

```
In [15]: plot_gallery("Faces from dataset", faces_centered[:n_components])
```

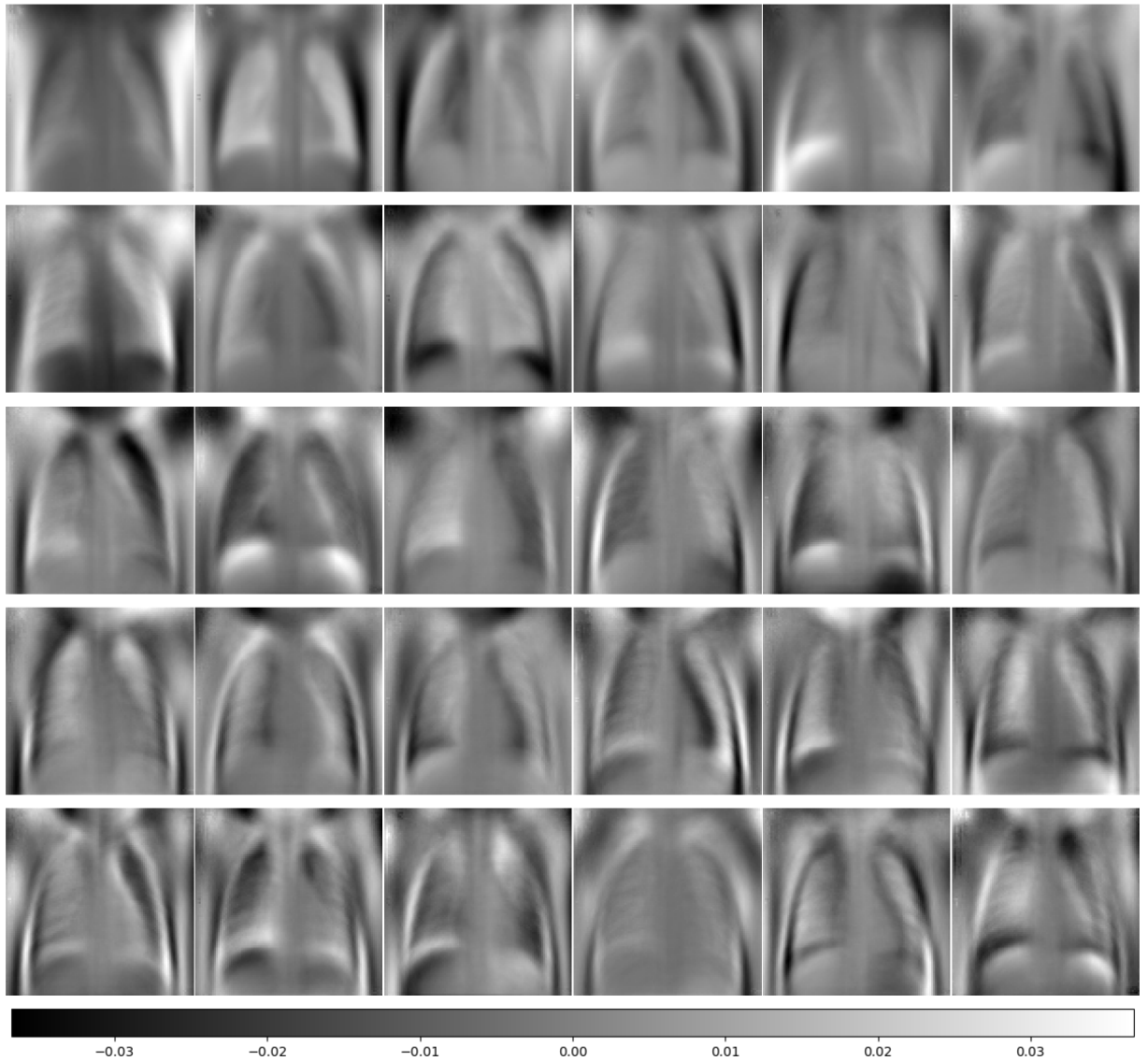

Faces from dataset



```
In [16]: pca_estimator = decomposition.PCA(
           n_components=n_components, svd_solver="auto", whiten=True
         )
pca_estimator.fit(faces_centered)

plot_gallery(
    "Eigenfaces - PCA", pca_estimator.components_[:n_components]
)
```

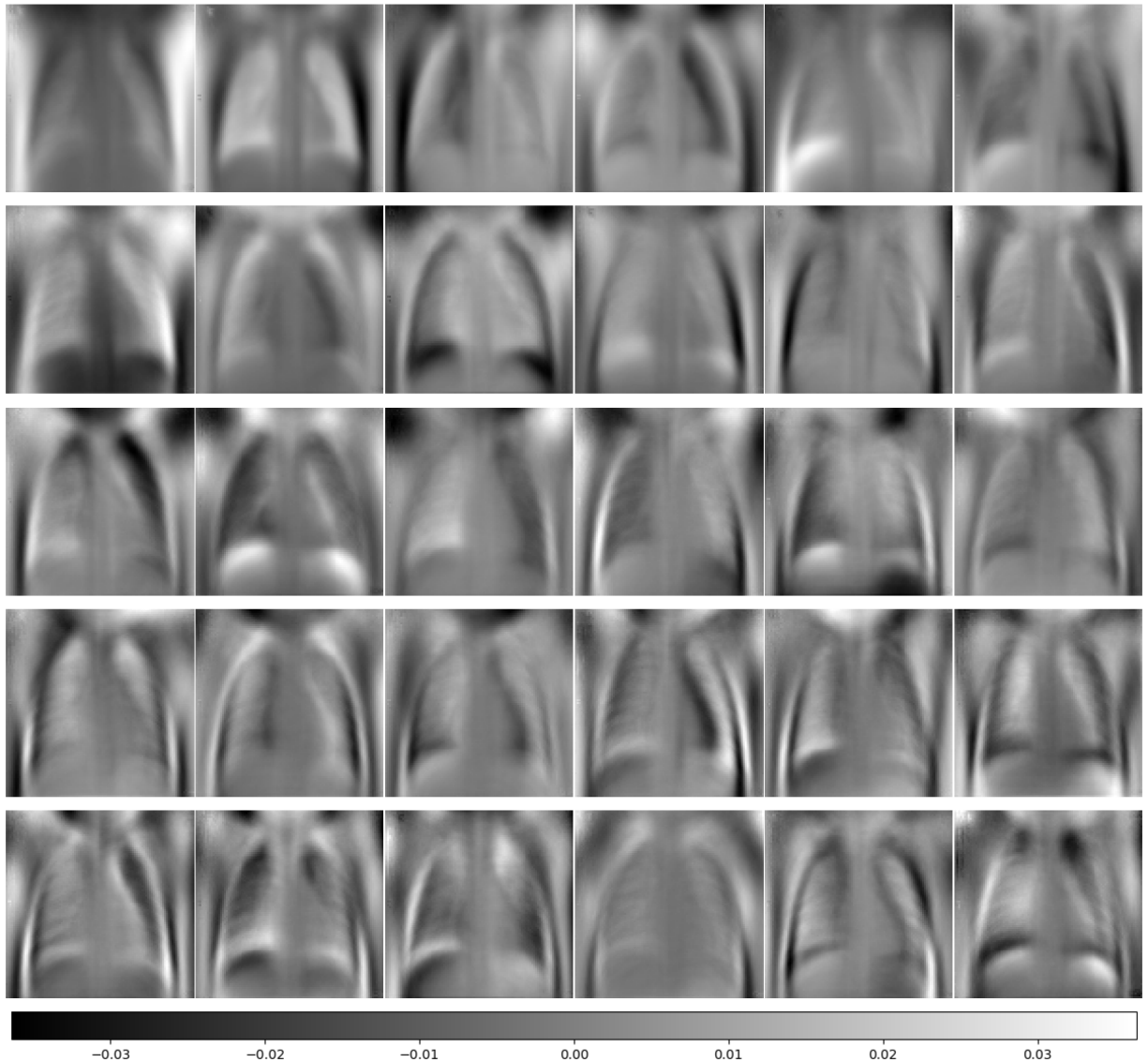
Eigenfaces - PCA



```
In [17]: pca_estimator = decomposition.PCA(
           n_components=n_components, svd_solver="randomized", whiten=True
         )
pca_estimator.fit(faces_centered)

plot_gallery(
    "Eigenfaces - PCA using randomized SVD", pca_estimator.components_[:n_comp]
)
```

Eigenfaces - PCA using randomized SVD



Cant see much being highlighted, maybe those with pneumonia have a wider spread of white noise than those with normal

PCA on Pneumonia Only

```
In [23]: # drop label to see only the image pixels
         grayscale_df_pneumonia_imageonly = grayscale_df_pneumonia_train.drop('label',

In [24]: # randomize dataset to view images later
         grayscale_df_pneumonia_imageonly = grayscale_df_pneumonia_imageonly.sample(fra
         grayscale_df_pneumonia_imageonly = grayscale_df_pneumonia_imageonly.drop('inde

In [25]: #turn image points to numpy
         grayscale_df_pneumonia_imageonly_array = grayscale_df_pneumonia_imageonly.to_n

         # Centering of data

         faces_centered = grayscale_df_pneumonia_imageonly_array - grayscale_df_pneumon
         faces_centered
```

```
Out[25]: array([[ 0.11110234,  0.11285111,  0.08726831, ..., -0.09171512,
                -0.08611765, -0.09276458],
                [-0.08105452, -0.07930576, -0.08920228, ...,  0.02593194,
                -0.08219608, -0.08099987],
                [ 0.25620038,  0.25794915,  0.24413106, ...,  0.47299077,
                0.38054902,  0.25625503],
                ...,
                [-0.05752511, -0.13028615, -0.1362611 , ...,  0.01416724,
                0.01976471,  0.01703934],
                [-0.19870158, -0.21263909, -0.23037875, ..., -0.09171512,
                -0.08611765, -0.09276458],
                [-0.14772119, -0.16558027, -0.18331992, ..., -0.03289159,
                -0.08611765, -0.0653136 ]])
```

```
In [26]: faces_centered -= faces_centered.mean(axis=1).reshape(3875, -1)

print(faces_centered.shape)

(3875, 10000)
```

```
In [27]: # looking at a few components

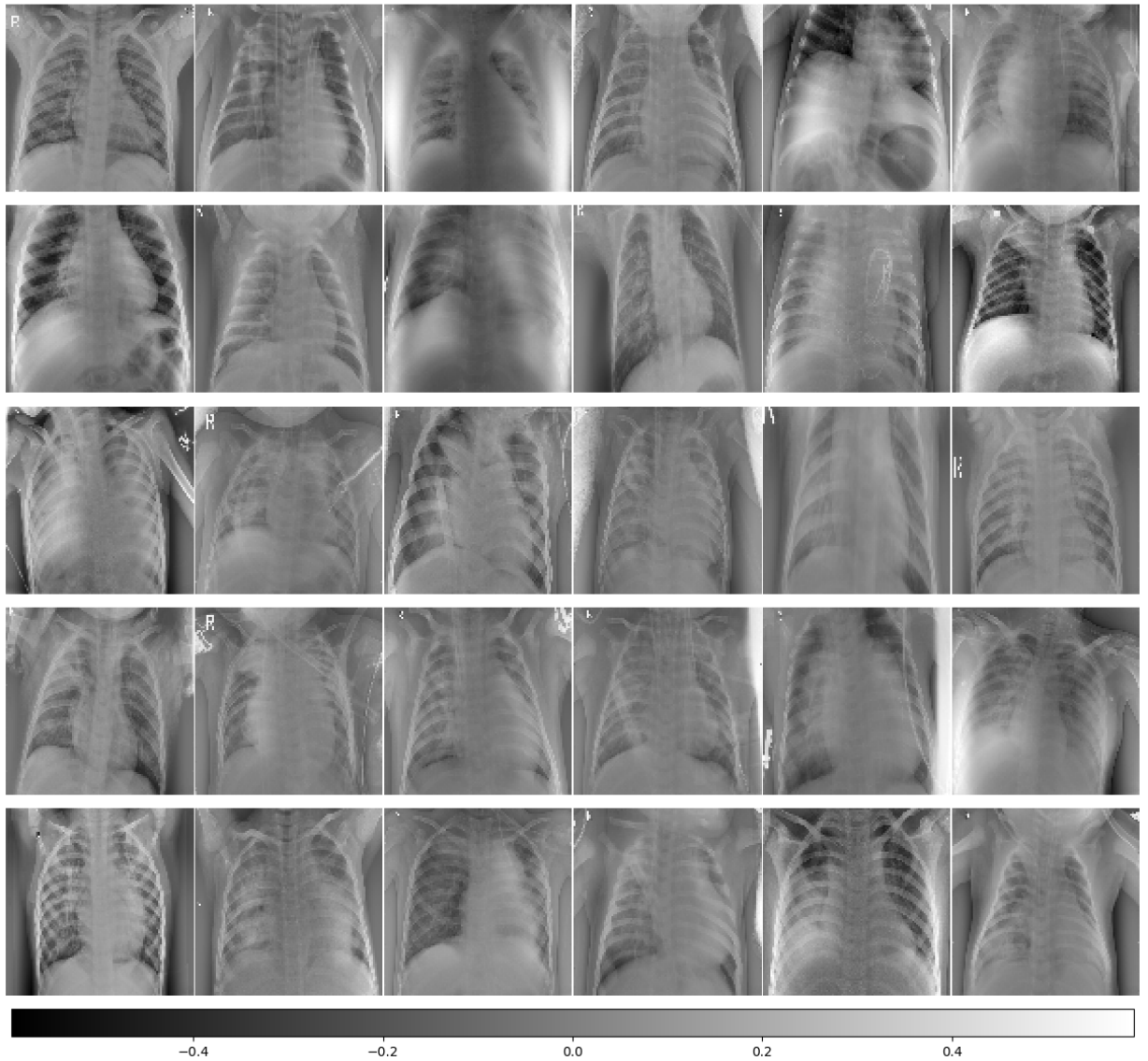
n_row, n_col = 5, 6
n_components = n_row * n_col
image_shape = (100, 100)

def plot_gallery(title, images, n_col=n_col, n_row=n_row, cmap=plt.cm.gray):
    fig, axs = plt.subplots(
        nrows=n_row,
        ncols=n_col,
        figsize=(2.0 * n_col, 2.3 * n_row),
        facecolor="white",
        constrained_layout=True,
    )
    fig.set_constrained_layout_pads(w_pad=0.01, h_pad=0.02, hspace=0, wspace=0)
    fig.set_edgecolor("black")
    fig.suptitle(title, size=16)
    for ax, vec in zip(axs.flat, images):
        vmax = max(vec.max(), -vec.min())
        im = ax.imshow(
            vec.reshape(image_shape),
            cmap=cmap,
            interpolation="nearest",
            vmin=-vmax,
            vmax=vmax,
        )
        ax.axis("off")

    fig.colorbar(im, ax=axs, orientation="horizontal", shrink=0.99, aspect=40,
plt.show()
```

```
In [28]: plot_gallery("Faces from dataset", faces_centered[:n_components])
```

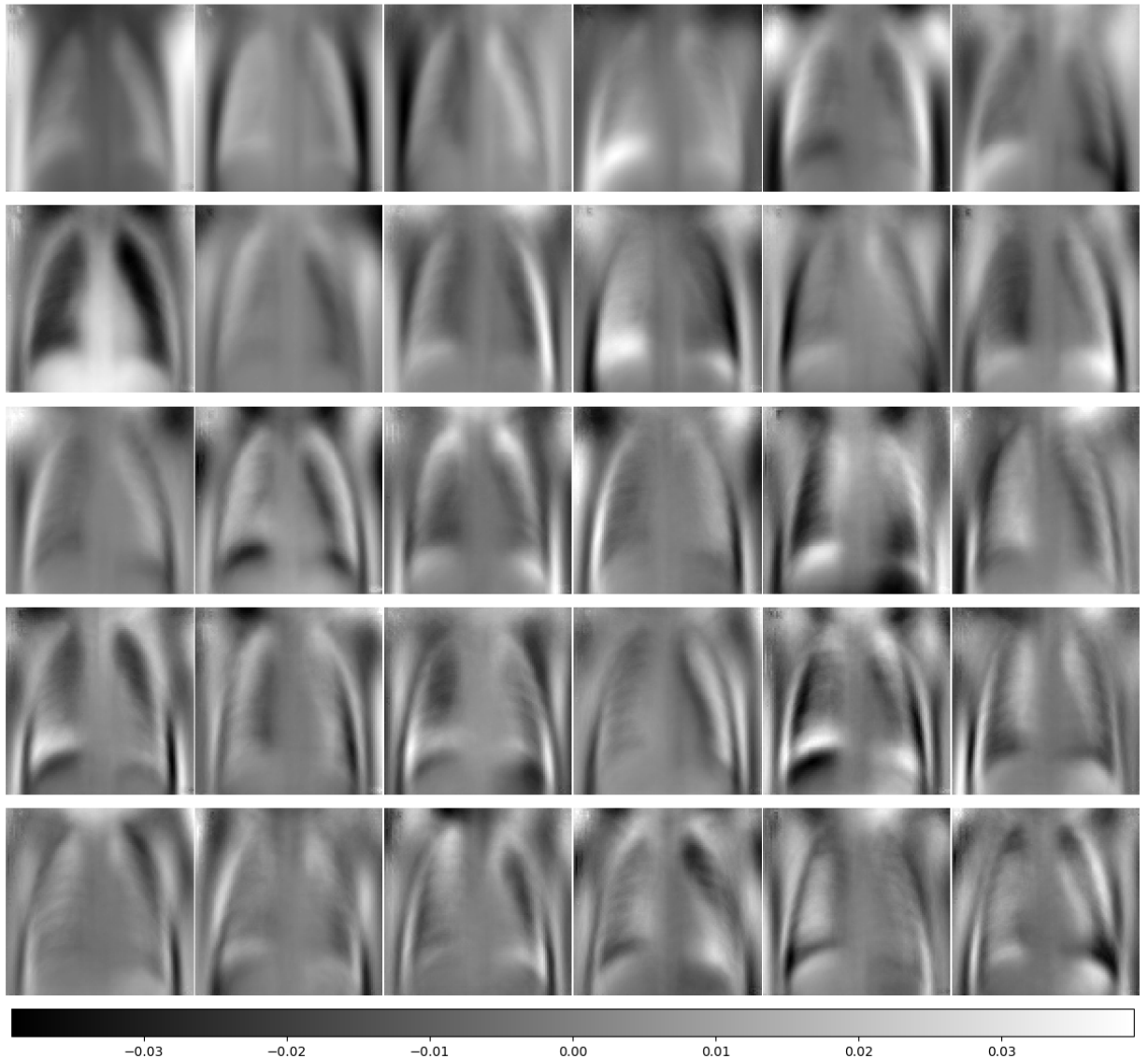
Faces from dataset



```
In [29]: pca_estimator = decomposition.PCA(
           n_components=n_components, svd_solver="randomized", whiten=True
         )
pca_estimator.fit(faces_centered)

plot_gallery(
    "Eigenfaces - PCA using randomized SVD", pca_estimator.components_[:n_comp
    )
```


Eigenfaces - PCA using randomized SVD



Do to the noise from pneumonia, cant really see much highlighted, mostly blurry, especially from the lungs

PCA on normal only

```
In [30]: # drop label to see only the image pixels
         grayscale_df_normal_imageonly = grayscale_df_normal_train.drop('label', axis=1)

In [31]: # randomize dataset to view images later
         grayscale_df_normal_imageonly = grayscale_df_normal_imageonly.sample(frac = 1)
         grayscale_df_normal_imageonly = grayscale_df_normal_imageonly.drop('index', axis=0)

In [32]: #turn image points to numpy
         grayscale_df_normal_imageonly_array = grayscale_df_normal_imageonly.to_numpy()

         # Centering of data

         faces_centered = grayscale_df_normal_imageonly_array - grayscale_df_normal_imageonly_array.mean(axis=0)
         faces_centered
```

```
Out[32]: array([[ -0.02853007,  0.00503575,  0.01896156, ..., -0.04040005,
                -0.02768785, -0.02381307],
                [-0.10696144, -0.14006229, -0.12613648, ..., -0.04040005,
                -0.02768785, -0.02381307],
                [-0.17362811, -0.20672896, -0.22809726, ..., -0.04040005,
                -0.02768785, -0.02381307],
                ...,
                [-0.17362811, -0.20672896, -0.22809726, ..., -0.04040005,
                -0.02768785, -0.02381307],
                [-0.17362811, -0.20672896, -0.18103844, ..., -0.04040005,
                -0.02768785, -0.02381307],
                [-0.17362811, -0.19496425, -0.14182275, ..., -0.04040005,
                -0.02768785, -0.02381307]])
```

```
In [33]: faces_centered -= faces_centered.mean(axis=1).reshape(1341, -1)

print(faces_centered.shape)

(1341, 10000)
```

```
In [34]: # looking at a few components

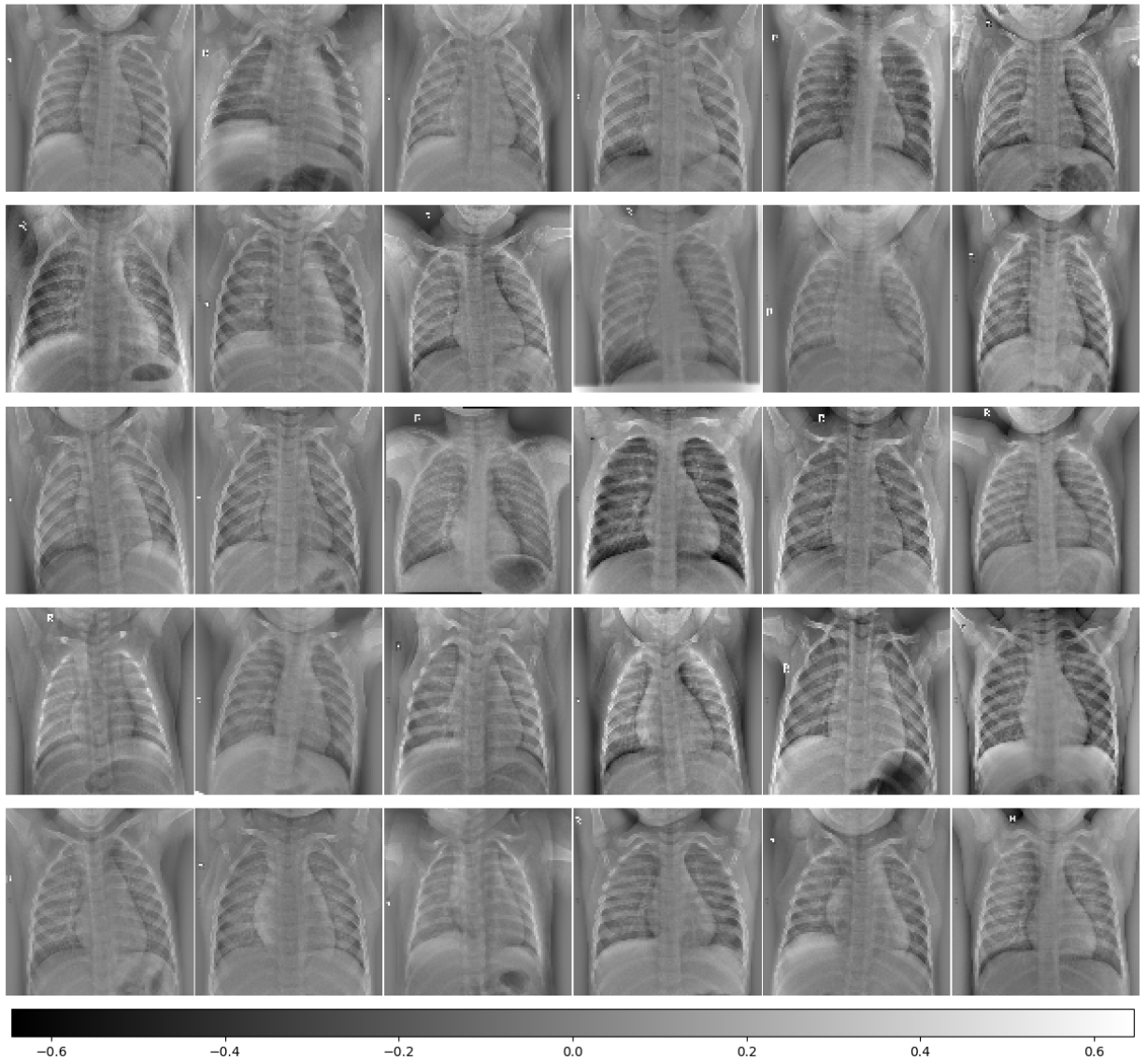
n_row, n_col = 5, 6
n_components = n_row * n_col
image_shape = (100, 100)

def plot_gallery(title, images, n_col=n_col, n_row=n_row, cmap=plt.cm.gray):
    fig, axs = plt.subplots(
        nrows=n_row,
        ncols=n_col,
        figsize=(2.0 * n_col, 2.3 * n_row),
        facecolor="white",
        constrained_layout=True,
    )
    fig.set_constrained_layout_pads(w_pad=0.01, h_pad=0.02, hspace=0, wspace=0)
    fig.set_edgecolor("black")
    fig.suptitle(title, size=16)
    for ax, vec in zip(axs.flat, images):
        vmax = max(vec.max(), -vec.min())
        im = ax.imshow(
            vec.reshape(image_shape),
            cmap=cmap,
            interpolation="nearest",
            vmin=-vmax,
            vmax=vmax,
        )
        ax.axis("off")

    fig.colorbar(im, ax=axs, orientation="horizontal", shrink=0.99, aspect=40,
plt.show()
```

```
In [35]: plot_gallery("Faces from dataset", faces_centered[:n_components])
```

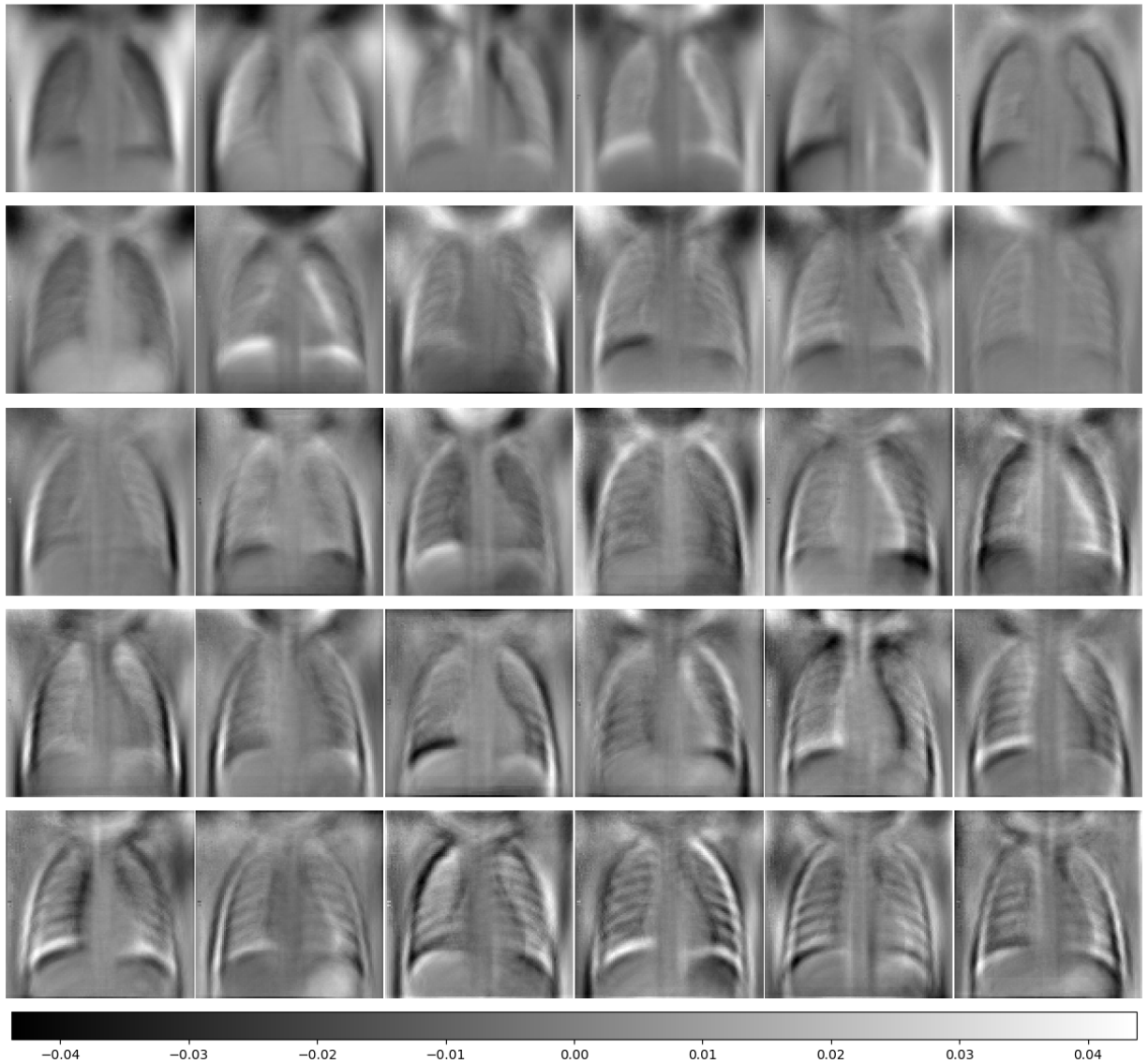
Faces from dataset



```
In [36]: pca_estimator = decomposition.PCA(
            n_components=n_components, svd_solver="randomized", whiten=True
        )
pca_estimator.fit(faces_centered)

plot_gallery(
    "Eigenfaces - PCA using randomized SVD", pca_estimator.components_[:n_comp
    ])
```


Eigenfaces - PCA using randomized SVD



You can see the bones and lungs better since they are highlighted

COMAPARING TSNE AND PCA, work in progress

```
In [37]: from sklearn.decomposition import PCA
```

```
In [38]: y = np.empty(5216, dtype=int)
count = 0
for labels in grayscale_df_train['label']:
    if labels == 'normal':
        y[count] = 0
    elif(labels == 'pneumonia'):
        y[count] = 1

    count = count + 1

print(y.shape)

(5216,)
```

```
In [39]: feat_cols = ['pixel'+str(i) for i in range( grayscale_df_train_imageonly.shape[1]
df = pd.DataFrame( grayscale_df_train_imageonly.values, columns=feat_cols)
df['label'] = y
print('Size of the dataframe: {}'.format( grayscale_df_train_imageonly.shape))
#X, y = None, None

#[out] Size of the dataframe: (733, 1002)
```

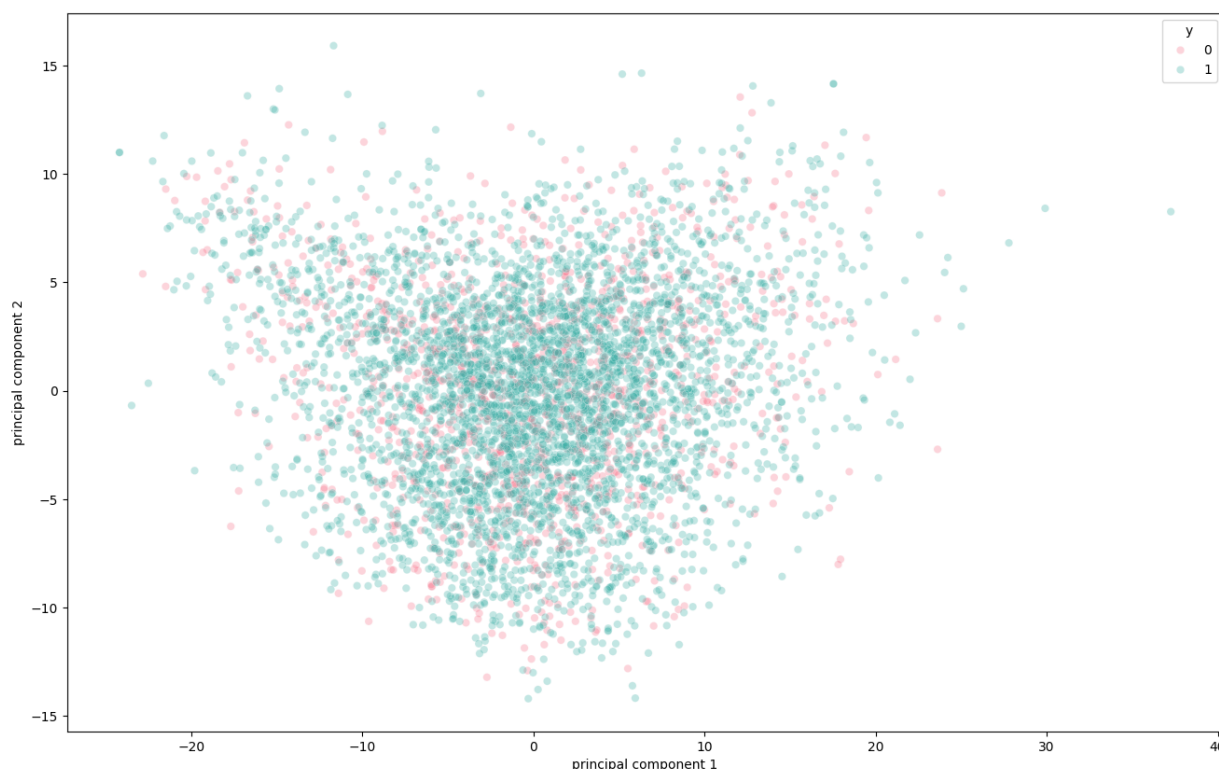
Size of the dataframe: (5216, 10000)

```
In [40]: # Another way of using PCA
pca_grayscale = PCA(n_components=3)
principalComponents_grayscale = pca_grayscale.fit_transform(df[feat_cols].values)
```

```
In [41]: principal_grayscale_Df = pd.DataFrame(data = principalComponents_grayscale,
, columns = ['principal component 1', 'principal component 2', 'principal component 3'])
principal_grayscale_Df['y'] = y
```

```
In [42]: import seaborn as sns
plt.figure(figsize=(16,10))
sns.scatterplot(
    x="principal component 1", y="principal component 2",
    hue="y",
    palette=sns.color_palette("husl", 2),
    data=principal_grayscale_Df,
    legend="full",
    alpha=0.3
)
```

Out[42]: <AxesSubplot: xlabel='principal component 1', ylabel='principal component 2'>



```
In [44]: from mpl_toolkits.mplot3d import Axes3D
```

```
In [46]: #fig = plt.figure(figsize=(6,6))
```

```

ax = plt.figure().add_subplot(projection='3d')#Axes3D(fig) # Method 1

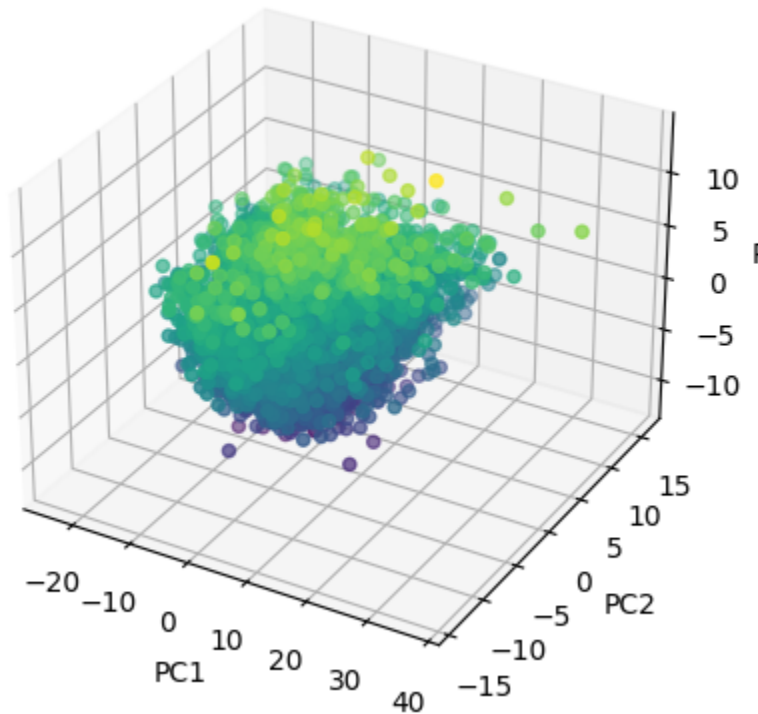
x = principal_grayscale_Df["principal component 1"]
y = principal_grayscale_Df["principal component 2"]
z = principal_grayscale_Df["principal component 3"]

ax.set_xlabel("PC1")
ax.set_ylabel("PC2")
ax.set_zlabel("PC3")

ax.scatter(x, y, z, c=z, marker='o')

plt.show()

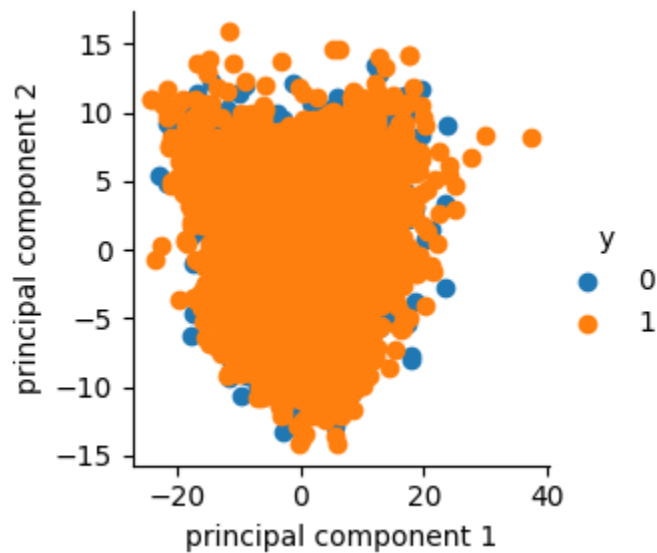
```



```
In [ ]: %matplotlib inline
```

```
In [47]: grid = sns.FacetGrid(principal_grayscale_Df, hue="y")
grid.map(plt.scatter, "principal component 1", "principal component 2").add_le
```

```
Out[47]: <seaborn.axisgrid.FacetGrid at 0x21801074790>
```



```
In [48]: grid = sns.FacetGrid(principal_grayscale_Df, col = "y", hue = "y", col_wrap=5)
grid.map(sns.scatterplot, "principal component 1", "principal component 2")

grid.add_legend()

plt.show()
```



```
In [49]: from sklearn.manifold import TSNE
```

```
In [62]: # For reproducibility of the results
np.random.seed(55)

rndperm = np.random.permutation(grayscale_df_train.shape[0])
```

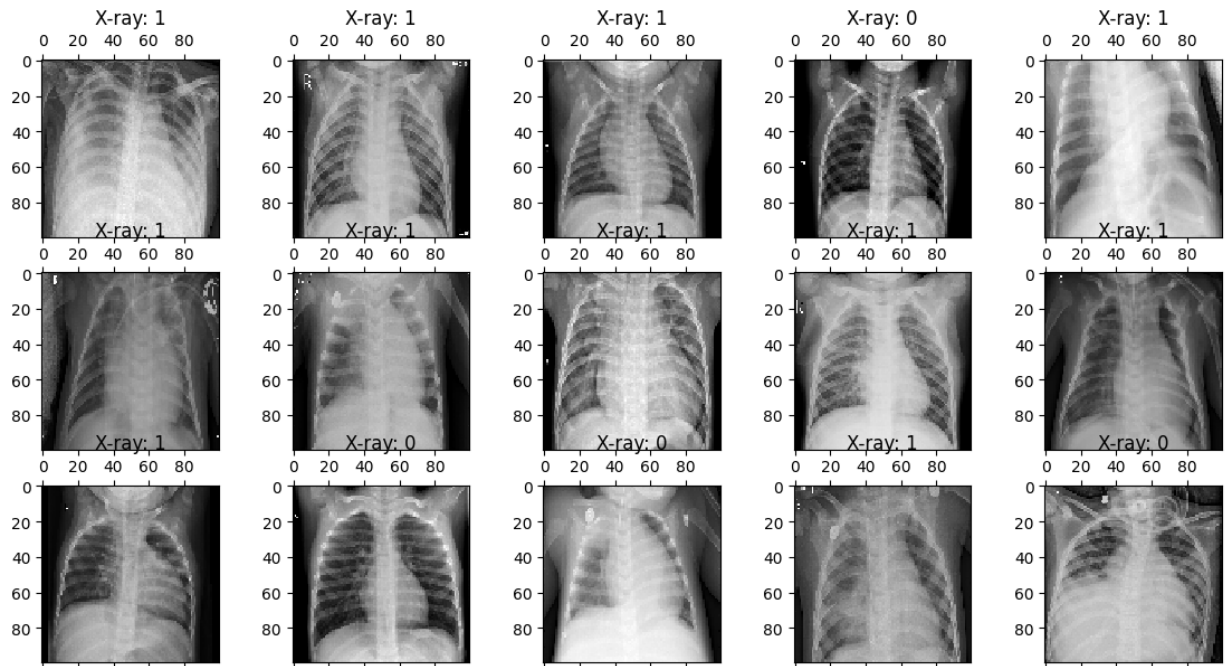
```
In [63]: rndperm
```

```
Out[63]: array([4537, 4208, 3551, ..., 4391, 4762, 4557])
```

```
In [64]: plt.gray()
fig = plt.figure( figsize=(14,7) )
for i in range(0,15):
    ax = fig.add_subplot(3,5,i+1, title="X-ray: {}".format(str(df.loc[rndperm[
    ax.matshow(df.loc[rndperm[i],feat_cols].values.reshape((100,100)).astype(f

plt.show()
```

<Figure size 640x480 with 0 Axes>



```
In [65]: pca = PCA(n_components=3)
pca_result = pca.fit_transform(df[feat_cols].values)

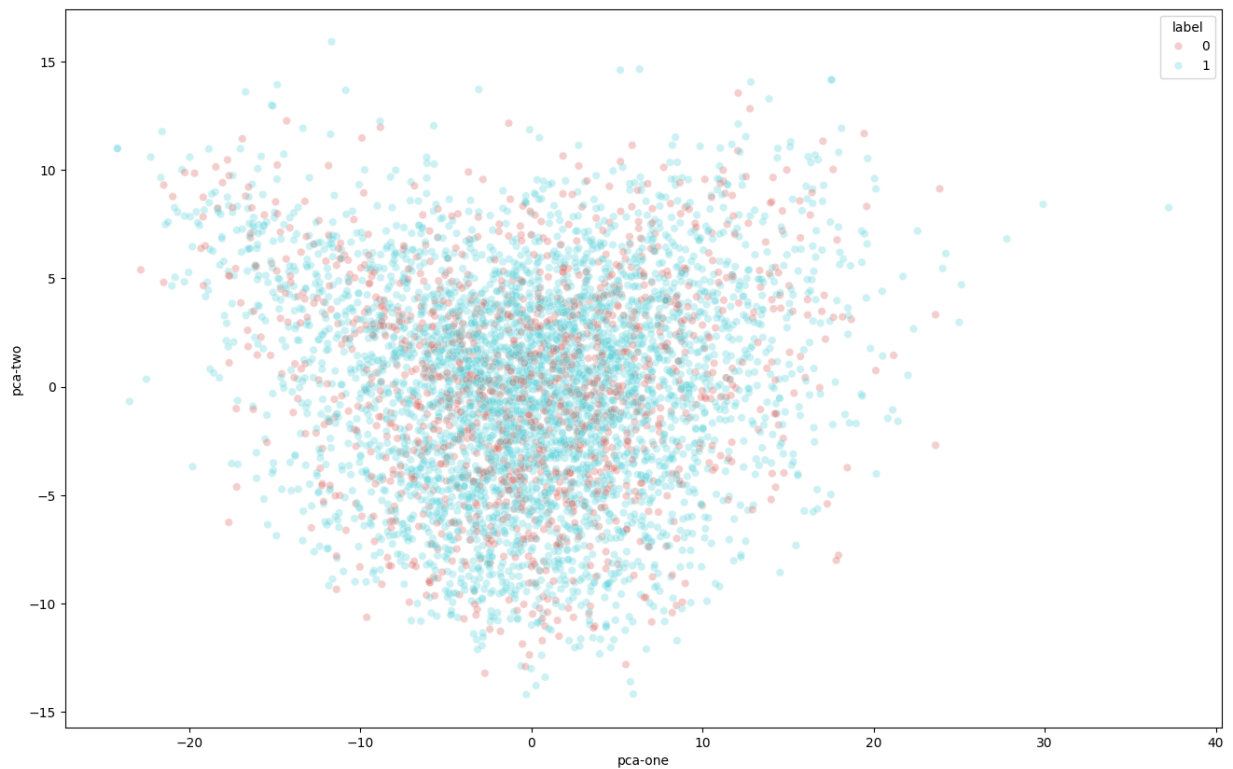
df['pca-one'] = pca_result[:,0]
df['pca-two'] = pca_result[:,1]
df['pca-three'] = pca_result[:,2]

print('Explained variation per principal component: {}'.format(pca.explained_v
```

Explained variation per principal component: [0.24935416 0.10293229 0.07142283]

```
In [66]: plt.figure(figsize=(16,10))
sns.scatterplot(
    x="pca-one", y="pca-two",
    hue="label",
    palette=sns.color_palette("hls", 2),
    data=df.loc[rndperm,:],
    legend="full",
    alpha=0.3
)
```

Out[66]: <AxesSubplot: xlabel='pca-one', ylabel='pca-two'>



```
In [67]: pca_50_2 = PCA(n_components=50)
pca_result_50 = pca_50_2.fit_transform(df[feat_cols].values)

print('Cumulative explained variation for 50 principal components: {}'.format(
Cumulative explained variation for 50 principal components: 0.8164541700755992
```

```
In [68]: time_start = time.time()

tsne = TSNE(n_components=2, verbose=0, perplexity=40, n_iter=300)
tsne_pca_results = tsne.fit_transform(pca_result_50)

print('t-SNE done! Time elapsed: {} seconds'.format(time.time()-time_start))
t-SNE done! Time elapsed: 14.549201011657715 seconds
```

```
In [71]: N = 5216

df_subset = df.loc[rndperm[:N],:].copy()

data_subset = df_subset[feat_cols].values
```

```
In [73]: time_start = time.time()
#tsne = TSNE(n_components=2, verbose=1, perplexity=40, n_iter=300)
tsne = TSNE(n_components=2, verbose=1, perplexity=10, n_iter=300)
tsne_results = tsne.fit_transform(data_subset)

print('t-SNE done! Time elapsed: {} seconds'.format(time.time()-time_start))

[t-SNE] Computing 31 nearest neighbors...
[t-SNE] Indexed 5216 samples in 0.864s...
[t-SNE] Computed neighbors for 5216 samples in 6.725s...
[t-SNE] Computed conditional probabilities for sample 1000 / 5216
[t-SNE] Computed conditional probabilities for sample 2000 / 5216
[t-SNE] Computed conditional probabilities for sample 3000 / 5216
[t-SNE] Computed conditional probabilities for sample 4000 / 5216
[t-SNE] Computed conditional probabilities for sample 5000 / 5216
```



```

[t-SNE] Computed conditional probabilities for sample 5216 / 5216
[t-SNE] Mean sigma: 3.018298
[t-SNE] KL divergence after 250 iterations with early exaggeration: 94.718491
[t-SNE] KL divergence after 300 iterations: 3.582578
t-SNE done! Time elapsed: 21.350158214569092 seconds

```

```

In [74]: df_subset['tsne-2d-one'] = tsne_results[:,0]
df_subset['tsne-2d-two'] = tsne_results[:,1]

plt.figure(figsize=(16,10))
sns.scatterplot(
    x="tsne-2d-one", y="tsne-2d-two",
    hue="label",
    palette=sns.color_palette("hls", 10),
    data=df_subset,
    legend="full",
    alpha=0.3
)

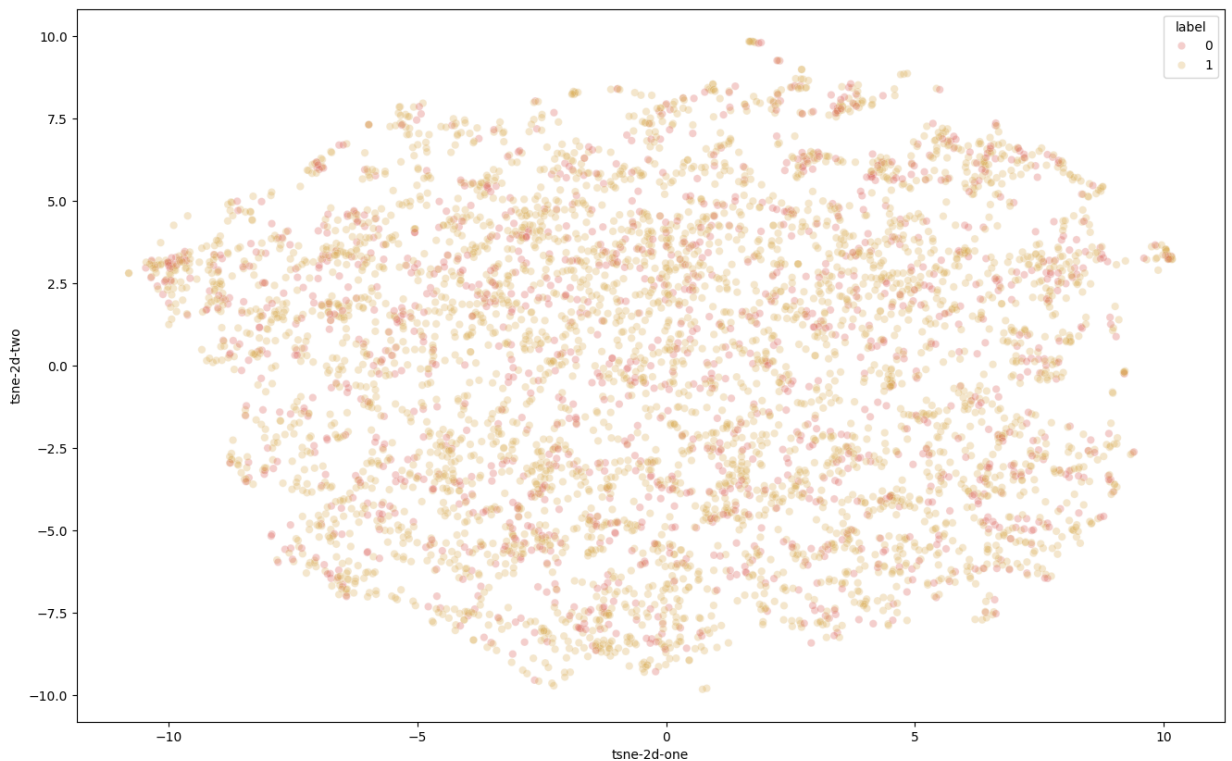
```

C:\Users\DellDepot\AppData\Local\Temp\ipykernel_19900\2544727987.py:5: UserWarning: The palette list has more values (10) than needed (2), which may not be intended.

```

Out[74]: <AxesSubplot: xlabel='tsne-2d-one', ylabel='tsne-2d-two'>

```



```

In [75]: df_subset['tsne-pca50-one'] = tsne_pca_results[:,0]
df_subset['tsne-pca50-two'] = tsne_pca_results[:,1]
plt.figure(figsize=(16,4))
ax1 = plt.subplot(1, 3, 1)
sns.scatterplot(
    x="pca-one", y="pca-two",
    hue="label",
    palette=sns.color_palette("hls", 10),
    data=df.loc[rndperm,:],
    legend="full",
    alpha=0.3
)

```

```

ax2 = plt.subplot(1, 3, 2)
sns.scatterplot(
    x="tsne-2d-one", y="tsne-2d-two",
    hue="label",
    palette=sns.color_palette("hls", 10),
    data=df_subset,
    legend=False,
    alpha=0.3,
    ax=ax2
)

ax3 = plt.subplot(1, 3, 3)
sns.scatterplot(
    x="tsne-pca50-one", y="tsne-pca50-two",
    hue="label",
    palette=sns.color_palette("hls", 10),
    data=df_subset,
    legend="full",
    alpha=0.3,
    ax=ax3
)

```

C:\Users\DellDepot\AppData\Local\Temp\ipykernel_19900\128147222.py:5: UserWarning: The palette list has more values (10) than needed (2), which may not be intended.

```
sns.scatterplot(
```

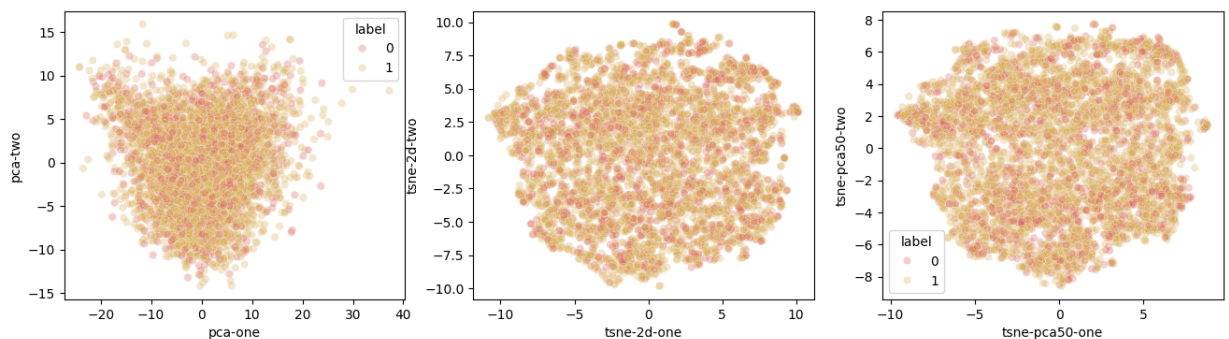
C:\Users\DellDepot\AppData\Local\Temp\ipykernel_19900\128147222.py:15: UserWarning: The palette list has more values (10) than needed (2), which may not be intended.

```
sns.scatterplot(
```

C:\Users\DellDepot\AppData\Local\Temp\ipykernel_19900\128147222.py:26: UserWarning: The palette list has more values (10) than needed (2), which may not be intended.

```
sns.scatterplot(
```

Out[75]: <AxesSubplot: xlabel='tsne-pca50-one', ylabel='tsne-pca50-two'>



In [76]: plt.figure(figsize=(16,7))

```

ax1 = plt.subplot(1, 2, 1)
sns.scatterplot(
    x="pca-one", y="pca-two",
    hue="label",
    palette=sns.color_palette("hls", 10),
    data=df.loc[rndperm,:],
    legend="full",
    alpha=0.3
)

ax2 = plt.subplot(1, 2, 2)

```



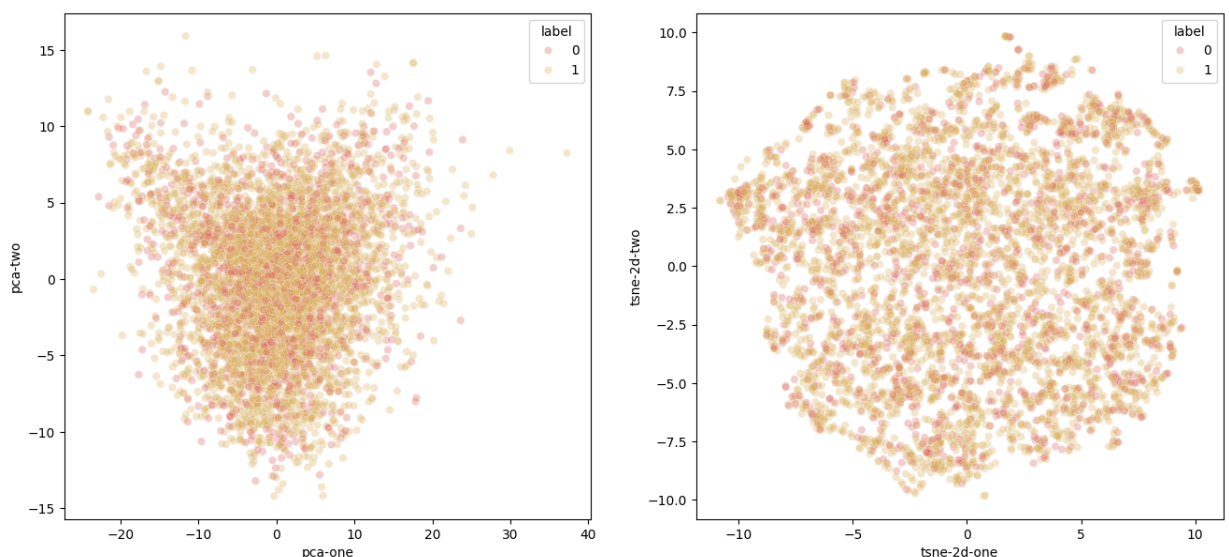
```
sns.scatterplot(
    x="tsne-2d-one", y="tsne-2d-two",
    hue="label",
    palette=sns.color_palette("hls", 10),
    data=df_subset,
    legend="full",
    alpha=0.3,
    ax=ax2
)
```

C:\Users\DellDepot\AppData\Local\Temp\ipykernel_19900\2966999904.py:4: UserWarning: The palette list has more values (10) than needed (2), which may not be intended.

C:\Users\DellDepot\AppData\Local\Temp\ipykernel_19900\2966999904.py:14: UserWarning: The palette list has more values (10) than needed (2), which may not be intended.

```
sns.scatterplot(
```

Out[76]: <AxesSubplot: xlabel='tsne-2d-one', ylabel='tsne-2d-two'>



Convolutional Neural Networks for Image Classification

Sources:

<https://medium.com/bitgrit-data-science-publication/building-an-image-classification-model-with-pytorch-from-scratch-f10452073212>

<https://www.pluralsight.com/guides/image-classification-with-pytorch>

```
In [163... import torch
import numpy as np
```

```
In [164... import torchvision
import torchvision.transforms as transforms
```

```
In [165... grayscale_df_train['label'].value_counts()
```

```
Out[165]: label
PNEUMONIA    3875
NORMAL        1341
Name: count, dtype: int64
```

```
In [166]: grayscale_df_test['label'].value_counts()
```

```
Out[166]: label
PNEUMONIA    390
NORMAL        234
Name: count, dtype: int64
```

```
In [167]: # Creating weights for each class due to unbalance train dataset
from sklearn.utils import class_weight

class_weights = class_weight.compute_class_weight(class_weight = 'balanced', c
class_weights=torch.tensor(class_weights,dtype=torch.float)

print(class_weights)

tensor([1.9448, 0.6730])
```

```
In [ ]: # Creating weights for each class due to unbalance train dataset
from sklearn.utils import class_weight

class_weights = class_weight.compute_class_weight(class_weight = 'balanced', c
class_weights=torch.tensor(class_weights,dtype=torch.float)

print(class_weights)
```

```
In [170]: grayscale_df_train
```

```
Out[170]:
```

	0	1	2	3	4	5	6	7	8	
0	0.086275	0.070588	0.094118	0.231373	0.278431	0.301961	0.345098	0.360784	0.372549	0
1	0.070588	0.121569	0.137255	0.168627	0.184314	0.192157	0.380392	0.372549	0.356863	0
2	0.000000	0.000000	0.105882	0.141176	0.176471	0.235294	0.282353	0.294118	0.301961	0
3	0.164706	0.200000	0.223529	0.274510	0.301961	0.321569	0.341176	0.388235	0.411765	0
4	0.317647	0.262745	0.211765	0.125490	0.109804	0.105882	0.117647	0.129412	0.133333	0
...	...	...	...	...	...	...	...	...	...	...
5211	0.000000	0.000000	0.000000	0.082353	0.156863	0.200000	0.231373	0.266667	0.286275	0
5212	0.364706	0.352941	0.337255	0.352941	0.435294	0.462745	0.352941	0.329412	0.341176	0
5213	0.101961	0.094118	0.007843	0.000000	0.000000	0.160784	0.176471	0.152941	0.196078	0
5214	0.125490	0.172549	0.137255	0.164706	0.215686	0.207843	0.176471	0.176471	0.168627	0
5215	0.054902	0.027451	0.023529	0.031373	0.043137	0.125490	0.121569	0.137255	0.149020	0

5216 rows × 10001 columns

USing CNN for Image Classification

This one has class_weight

Without any cahnges;

Accuracy for class: NORMAL is 36.3 % Accuracy for class: PNEUMONIA is 99.7 %

```
In [171... import torch
from torchvision import transforms, datasets

data_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5, 0.5, 0.5],
                          std=[0.5, 0.5, 0.5]),
    transforms.Resize([100,100])
])
xray_dataset = datasets.ImageFolder(root='C:/Users/DellDepot/Downloads/chest_x
                                     transform=data_transform)
train_loader = torch.utils.data.DataLoader(xray_dataset,
                                           batch_size=4, shuffle=True,
                                           num_workers=4)

### change num_workers to 0
```

```
In [172... import torch
import torchvision
import torchvision.transforms as transforms
```

```
In [173... import matplotlib.pyplot as plt
import numpy as np

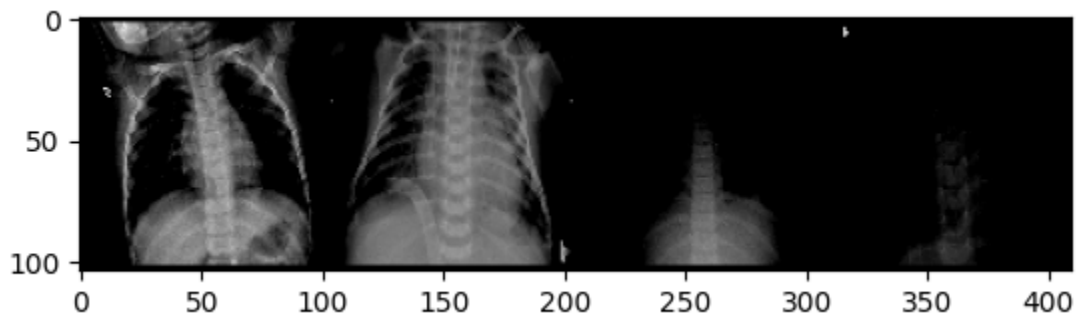
batch_size = 4

def imshow(img):
    npimg = img.numpy()
    plt.imshow(np.transpose(npimg, (1, 2, 0)))

# get some random training images
dataiter = iter(train_loader)
print("before next(dataiter):",dataiter)
type(dataiter)
images, labels = next(dataiter)
#print("images:",images)
print("labels:",labels)

classes = ['NORMAL', 'PNEUMONIA']
# show images
imshow(torchvision.utils.make_grid(images))
# print labels
print(' '.join(f'{classes[labels[j]]:5s}' for j in range(batch_size)))

before next(dataiter): <torch.utils.data.dataloader._MultiProcessingDataLoader
Iter object at 0x000002198DE6DA90>
Clipping input data to the valid range for imshow with RGB data ([0..1] for fl
oats or [0..255] for integers).
labels: tensor([0, 1, 1, 1])
NORMAL PNEUMONIA PNEUMONIA PNEUMONIA
```



```
In [174... import torch.nn as nn
import torch.nn.functional as F

class Net(nn.Module):
    def __init__(self):
        super().__init__()

        # Notes: What does nn.Conv2d do?
        # nn.Conv2d(in channel, out channel, kernel_size)
        # in_channel = 3 because of RGB
        # out_channel = 6 features
        # kernel_size = size of kernel (acts like a feature detector throughout)
        self.conv1 = nn.Conv2d(3, 6, 5)
        # converts image 3 * 100 * 100 to image 6 * 96 * 96
        self.pool = nn.MaxPool2d(2, 2)
        # converts image 6 * 96 * 96 to image 6 * 48 * 48
        self.conv2 = nn.Conv2d(6, 16, 5)
        # converts image 6 * 48 * 48 to image 16 * 44 * 44
        self.fc1 = nn.Linear(16 * 22 * 22, 44 * 44) #
        self.fc2 = nn.Linear(44 * 44, 20 * 20)
        self.fc3 = nn.Linear(20 * 20, 2) #400 by 2

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = torch.flatten(x, 1)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x

net = Net()
```

```
In [175... import torch.optim as optim

criterion = nn.CrossEntropyLoss(weight=class_weights, reduction='mean')
optimizer = optim.SGD(net.parameters(), lr=0.001, momentum=0.9)
```

```
In [176... len(train_loader)
```

```
Out[176]: 1304
```

```
In [177... for epoch in range(3): # loop over the dataset multiple times

    running_loss = 0.0
    for i, data in enumerate(train_loader, 0):
```

```

# get the inputs; data is a list of [inputs, labels]
inputs, labels = data

# zero the parameter gradients
optimizer.zero_grad()

# forward + backward + optimize
outputs = net(inputs)
# print(outputs)
loss = criterion(outputs, labels)
loss.backward()
optimizer.step()

# print statistics
running_loss += loss.item()
if i % 200 == 199:    # print every 200 mini-batches
    print(f'[{epoch + 1}, {i + 1:5d}] loss: {running_loss / 200:.3f}')
    running_loss = 0.0

print('Finished Training')

```

[1, 200] loss: 0.660
 [1, 400] loss: 0.274
 [1, 600] loss: 0.267
 [1, 800] loss: 0.219
 [1, 1000] loss: 0.170
 [1, 1200] loss: 0.182
 [2, 200] loss: 0.139
 [2, 400] loss: 0.113
 [2, 600] loss: 0.143
 [2, 800] loss: 0.144
 [2, 1000] loss: 0.158
 [2, 1200] loss: 0.167
 [3, 200] loss: 0.140
 [3, 400] loss: 0.139
 [3, 600] loss: 0.117
 [3, 800] loss: 0.070
 [3, 1000] loss: 0.122
 [3, 1200] loss: 0.096
 Finished Training

```

In [178... # save
PATH = './xray_net.pth'
torch.save(net.state_dict(), PATH)
# reload
net = Net()
net.load_state_dict(torch.load(PATH))

```

Out[178]: <All keys matched successfully>

```

In [179... xray_testdataset = datasets.ImageFolder(root='C:/Users/DellDepot/Downloads/che
transform=data_transform)
test_loader = torch.utils.data.DataLoader(xray_testdataset,
batch_size=4, shuffle=True,
num_workers=4)

```

```

In [180... dataiter = iter(test_loader)
images, labels = next(dataiter)

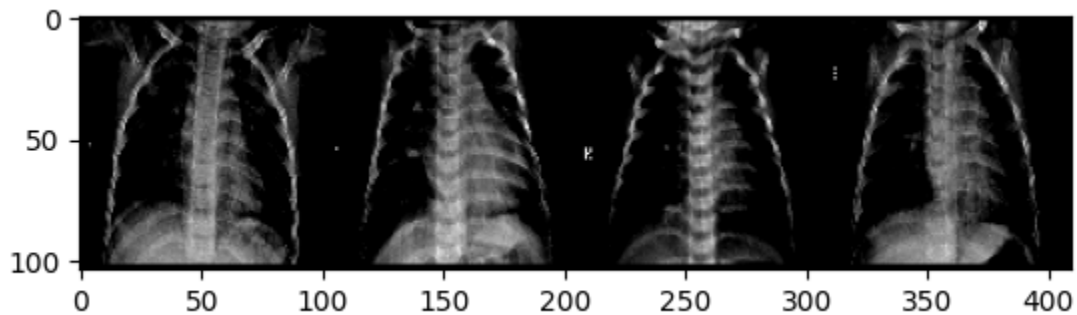
# print images

```

```
imshow(torchvision.utils.make_grid(images))
print('GroundTruth: ', ' '.join(f'{classes[labels[j]]:5s}' for j in range(4)))
```

Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).

GroundTruth: NORMAL PNEUMONIA PNEUMONIA NORMAL



```
In [181...] outputs = net(images)
```

```
In [182...] _, predicted = torch.max(outputs, 1)
print('Predicted: ', ' '.join(f'{classes[predicted[j]]:5s}'
                              for j in range(4)))
```

Predicted: PNEUMONIA PNEUMONIA PNEUMONIA PNEUMONIA

```
In [183...] correct = 0
total = 0

with torch.no_grad():
    for data in test_loader:
        images, labels = data
        outputs = net(images)
        _, predicted = torch.max(outputs.data, 1)
        #print(predicted, labels)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

print(f'Accuracy of the network on the 10000 test images: {100 * correct // total} %')
print(total)
print(labels)
```

Accuracy of the network on the 10000 test images: 74 %
624
tensor([1, 0, 0, 0])

```
In [184...] correct_pred = {classname: 0 for classname in classes}
total_pred = {classname: 0 for classname in classes}

with torch.no_grad():
    for data in test_loader:
        images, labels = data
        outputs = net(images)
        _, predictions = torch.max(outputs, 1)
        for label, prediction in zip(labels, predictions):
            if label == prediction:
                correct_pred[classes[label]] += 1
                total_pred[classes[label]] += 1

    for classname, correct_count in correct_pred.items():
```

```
accuracy = 100 * float(correct_count) / total_pred[classname]
print(f'Accuracy for class: {classname:5s} is {accuracy:.1f} %')
```

Accuracy for class: NORMAL is 32.5 %
Accuracy for class: PNEUMONIA is 99.5 %

In [53]: *# Create confusion matrix*

In []: Try different hyperparameters, particularly **in** learning rate
Try_batch size = 32
Try training **for** longer epochs
Try a different optimizer (like adam)

Try different hyperparameter

hyperparameter

learning rate (lr = 1e-4)

In [54]: **import** torch.optim **as** optim

```
criterion = nn.CrossEntropyLoss(weight=class_weights,reduction='mean')
optimizer = optim.SGD(net.parameters(), lr=0.0001, momentum=0.9)
```

In [55]: **for** epoch **in** range(3): *# loop over the dataset multiple times*

```
    running_loss = 0.0
    for i, data in enumerate(train_loader, 0):
        # get the inputs; data is a list of [inputs, labels]
        inputs, labels = data

        # zero the parameter gradients
        optimizer.zero_grad()

        # forward + backward + optimize
        outputs = net(inputs)
        # print(outputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        # print statistics
        running_loss += loss.item()
        if i % 200 == 199: # print every 200 mini-batches
            print(f'[{epoch + 1}, {i + 1:5d}] loss: {running_loss / 200:.3f}')
            running_loss = 0.0

    print('Finished Training')
```

```
[1, 200] loss: 0.095
[1, 400] loss: 0.096
[1, 600] loss: 0.099
[1, 800] loss: 0.071
[1, 1000] loss: 0.094
[1, 1200] loss: 0.053
[2, 200] loss: 0.094
[2, 400] loss: 0.087
[2, 600] loss: 0.085
[2, 800] loss: 0.050
```



```

[2, 1000] loss: 0.067
[2, 1200] loss: 0.063
[3, 200] loss: 0.068
[3, 400] loss: 0.071
[3, 600] loss: 0.072
[3, 800] loss: 0.055
[3, 1000] loss: 0.071
[3, 1200] loss: 0.069
Finished Training

```

```

In [56]: # save
PATH = './xray_net.pth'
torch.save(net.state_dict(), PATH)
# reload
net = Net()
net.load_state_dict(torch.load(PATH))

```

Out[56]: <All keys matched successfully>

```

In [57]: xray_testdataset = datasets.ImageFolder(root='C:/Users/DellDepot/Downloads/che
transform=data_transform)
test_loader = torch.utils.data.DataLoader(xray_testdataset,
batch_size=4, shuffle=True,
num_workers=4)

```

```

In [58]: dataiter = iter(test_loader)
images, labels = next(dataiter)

# print images
imshow(torchvision.utils.make_grid(images))
print('GroundTruth: ', ' '.join('%5s' % classes[labels[j]] for j in range(4)))

```

Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).

GroundTruth: PNEUMONIA NORMAL NORMAL PNEUMONIA



```

In [59]: outputs = net(images)

```

```

In [60]: _, predicted = torch.max(outputs, 1)
print('Predicted: ', ' '.join('%5s' % classes[predicted[j]]
for j in range(4)))

```

Predicted: PNEUMONIA NORMAL NORMAL PNEUMONIA

```

In [61]: correct = 0
total = 0

with torch.no_grad():
for data in test_loader:

```

```

        images, labels = data
        outputs = net(images)
        _, predicted = torch.max(outputs.data, 1)
        #print(predicted, labels)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

print(f'Accuracy of the network on the 10000 test images: {100 * correct // to
print(total)
print(labels)

```

Accuracy of the network on the 10000 test images: 78 %
624
tensor([1, 0, 0, 0])

In [62]:

```

correct_pred = {classname: 0 for classname in classes}
total_pred = {classname: 0 for classname in classes}

with torch.no_grad():
    for data in test_loader:
        images, labels = data
        outputs = net(images)
        _, predictions = torch.max(outputs, 1)
        for label, prediction in zip(labels, predictions):
            if label == prediction:
                correct_pred[classes[label]] += 1
                total_pred[classes[label]] += 1

for classname, correct_count in correct_pred.items():
    accuracy = 100 * float(correct_count) / total_pred[classname]
    print(f'Accuracy for class: {classname:5s} is {accuracy:.1f} %')

```

Accuracy for class: NORMAL is 46.2 %
Accuracy for class: PNEUMONIA is 97.9 %

It did not really change the accuracy,

Lets try a bigger batch size = 32, keep learning rate and class weight

In [185...]

```

import torch
from torchvision import transforms, datasets

data_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5, 0.5, 0.5],
                          std=[0.5, 0.5, 0.5]),
    transforms.Resize([100,100])
])
xray_dataset = datasets.ImageFolder(root='C:/Users/DellDepot/Downloads/chest_x
                                transform=data_transform)
train_loader = torch.utils.data.DataLoader(xray_dataset,
                                           batch_size=32, shuffle=True,
                                           num_workers=4)

```

In [186...]

```

import matplotlib.pyplot as plt
import numpy as np

batch_size = 32

def imshow(img):
    npimg = img.numpy()

```

```
plt.imshow(np.transpose(npimg, (1, 2, 0)))

# get some random training images
dataiter = iter(train_loader)
print("before next(dataiter):", dataiter)
type(dataiter)
images, labels = next(dataiter)
#print("images:", images)
print("labels:", labels)

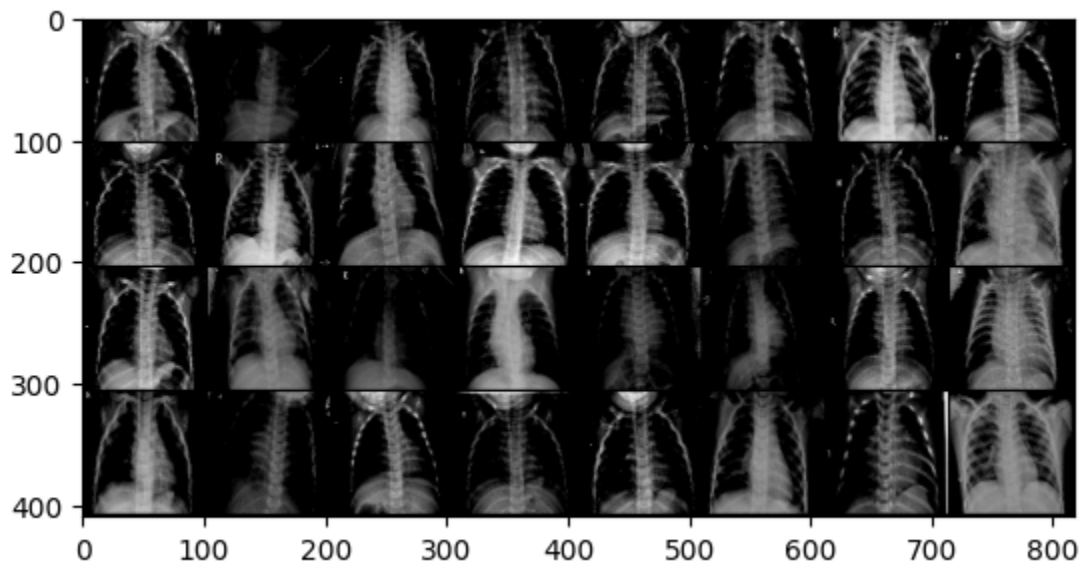
classes = ['NORMAL', 'PNEUMONIA']
# show images
imshow(torchvision.utils.make_grid(images))
# print labels
print(' '.join(f'{classes[labels[j]]:5s}' for j in range(batch_size)))
```

before next(dataiter): <torch.utils.data.dataloader._MultiProcessingDataLoaderIter object at 0x00000218556B5190>

Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).

labels: tensor([0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 0, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 0, 0, 1, 1, 1])

NORMAL PNEUMONIA PNEUMONIA PNEUMONIA NORMAL PNEUMONIA PNEUMONIA NORMAL NORMAL
PNEUMONIA PNEUMONIA NORMAL NORMAL PNEUMONIA NORMAL PNEUMONIA NORMAL PNEUMONIA
PNEUMONIA PNEUMONIA PNEUMONIA PNEUMONIA NORMAL PNEUMONIA PNEUMONIA PNEUMONIA N
ORMAL NORMAL NORMAL PNEUMONIA PNEUMONIA PNEUMONIA



```
In [187... import torch.optim as optim

criterion = nn.CrossEntropyLoss(weight=class_weights, reduction='mean')
optimizer = optim.SGD(net.parameters(), lr=0.0001, momentum=0.9)
```

```
In [188... for epoch in range(3): # loop over the dataset multiple times

    running_loss = 0.0
    for i, data in enumerate(train_loader, 0):
        # get the inputs; data is a list of [inputs, labels]
        inputs, labels = data

        # zero the parameter gradients
        optimizer.zero_grad()
```

```

        # forward + backward + optimize
        outputs = net(inputs)
        # print(outputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        # print statistics
        running_loss += loss.item()
        if i % 50 == 49:    # print every 50 mini-batches
            print(f'[{epoch + 1}, {i + 1:5d}] loss: {running_loss / 200:.3f}')
            running_loss = 0.0

print('Finished Training')

[1,   50] loss: 0.023
[1,  100] loss: 0.020
[1,  150] loss: 0.022
[2,   50] loss: 0.017
[2,  100] loss: 0.021
[2,  150] loss: 0.019
[3,   50] loss: 0.018
[3,  100] loss: 0.022
[3,  150] loss: 0.017
Finished Training

```

In [189]: `len(train_loader)`

Out[189]: 163

```

In [190]: # save
PATH = './xray_net.pth'
torch.save(net.state_dict(), PATH)
# reload
net = Net()
net.load_state_dict(torch.load(PATH))

```

Out[190]: <All keys matched successfully>

```

In [191]: xray_testdataset = datasets.ImageFolder(root='C:/Users/DellDepot/Downloads/chest_xray',
transform=data_transform)
test_loader = torch.utils.data.DataLoader(xray_testdataset,
batch_size=32, shuffle=True,
num_workers=4)

```

```

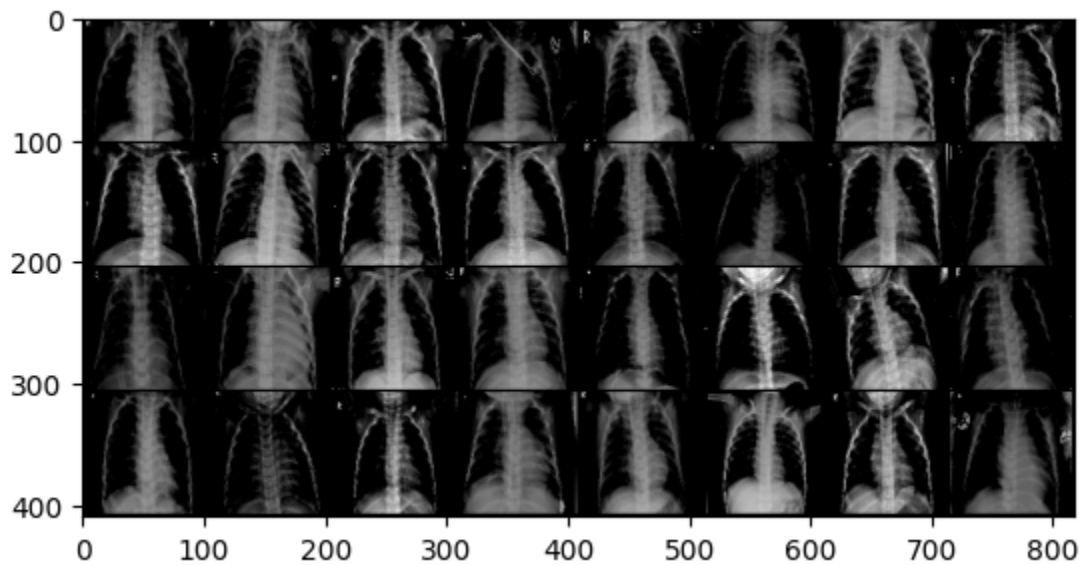
In [192]: dataiter = iter(test_loader)
images, labels = next(dataiter)

# print images
imshow(torchvision.utils.make_grid(images))
print('GroundTruth: ', ' '.join('%5s' % classes[labels[j]] for j in range(32)))

```

Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).

GroundTruth: PNEUMONIA PNEUMONIA NORMAL PNEUMONIA PNEUMONIA PNEUMONIA NORMAL
NORMAL NORMAL PNEUMONIA NORMAL NORMAL PNEUMONIA PNEUMONIA NORMAL PNEUMONIA PNEUMONIA
PNEUMONIA PNEUMONIA PNEUMONIA PNEUMONIA PNEUMONIA NORMAL NORMAL PNEUMONIA PNEUMONIA
NORMAL NORMAL PNEUMONIA PNEUMONIA NORMAL NORMAL PNEUMONIA



```
In [193...] outputs = net(images)
```

```
In [194...] _, predicted = torch.max(outputs, 1)

print('Predicted: ', ' '.join(f'{classes[predicted[j]]:5s}'
                              for j in range(32)))
```

Predicted: PNEUMONIA PNEUMONIA NORMAL PNEUMONIA PNEUMONIA PNEUMONIA PNEUMONIA PNEUMONIA
 NORMAL PNEUMONIA PNEUMONIA PNEUMONIA PNEUMONIA PNEUMONIA PNEUMONIA PNEUMONIA P
 NEUMONIA PNEUMONIA PNEUMONIA PNEUMONIA PNEUMONIA PNEUMONIA PNEUMONIA NORMAL NORMAL PNEUM
 ONIA PNEUMONIA PNEUMONIA NORMAL PNEUMONIA PNEUMONIA NORMAL NORMAL PNEUMONIA

```
In [195...] correct = 0
total = 0

with torch.no_grad():
    for data in test_loader:
        images, labels = data
        outputs = net(images)
        _, predicted = torch.max(outputs.data, 1)
        #print(predicted, labels)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

print(f'Accuracy of the network on the 10000 test images: {100 * correct // to
print(total)
print(labels)
```

Accuracy of the network on the 10000 test images: 81 %
 624
 tensor([0, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 0])

```
In [196...] correct_pred = {classname: 0 for classname in classes}
total_pred = {classname: 0 for classname in classes}

with torch.no_grad():
    for data in test_loader:
        images, labels = data
        outputs = net(images)
        _, predictions = torch.max(outputs, 1)
        for label, prediction in zip(labels, predictions):
            if label == prediction:
```

```

        correct_pred[classes[label]] += 1
        total_pred[classes[label]] += 1

for classname, correct_count in correct_pred.items():
    accuracy = 100 * float(correct_count) / total_pred[classname]
    print(f'Accuracy for class: {classname:5s} is {accuracy:.1f} %')

```

Accuracy for class: NORMAL is 53.8 %
Accuracy for class: PNEUMONIA is 97.9 %

In []: *# no much changes either*

In []: *# Let us try more epochs, with batch size = 32, lr = 0.0001, etc and weight cl*

In [77]:

```

import torch
from torchvision import transforms, datasets

data_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5, 0.5, 0.5],
                          std=[0.5, 0.5, 0.5]),
    transforms.Resize([100,100])
])
xray_dataset = datasets.ImageFolder(root='C:/Users/DellDepot/Downloads/chest_x
                                   transform=data_transform)
train_loader = torch.utils.data.DataLoader(xray_dataset,
                                           batch_size=32, shuffle=True,
                                           num_workers=4)

```

In [78]:

```

import matplotlib.pyplot as plt
import numpy as np

batch_size = 32

def imshow(img):
    npimg = img.numpy()
    plt.imshow(np.transpose(npimg, (1, 2, 0)))

# get some random training images
dataiter = iter(train_loader)
print("before next(dataiter):", dataiter)
type(dataiter)
images, labels = next(dataiter)
#print("images:", images)
print("labels:", labels)

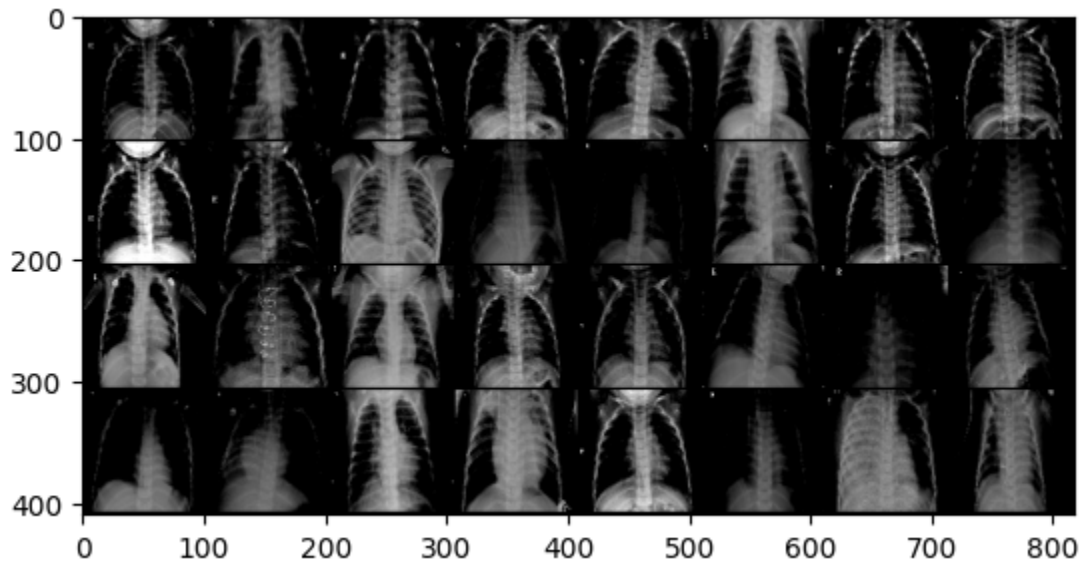
classes = ['NORMAL', 'PNEUMONIA']
# show images
imshow(torchvision.utils.make_grid(images))
# print labels
print(' '.join(f'{classes[labels[j]]:5s}' for j in range(batch_size)))

before next(dataiter): <torch.utils.data.dataloader._MultiProcessingDataLoader
Iter object at 0x0000025D2EB1FFA0>
labels: tensor([0, 1, 1, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 0,
1, 1, 1,
              1, 1, 1, 1, 0, 1, 1, 1])

Clipping input data to the valid range for imshow with RGB data ([0..1] for fl
oats or [0..255] for integers).

```

NORMAL PNEUMONIA PNEUMONIA NORMAL PNEUMONIA PNEUMONIA PNEUMONIA NORMAL NORMAL
 PNEUMONIA PNEUMONIA PNEUMONIA PNEUMONIA PNEUMONIA NORMAL PNEUMONIA PNEUMONIA
 PNEUMONIA PNEUMONIA NORMAL NORMAL PNEUMONIA PNEUMONIA PNEUMONIA PNEUMONIA PNEUM
 ONIA PNEUMONIA PNEUMONIA NORMAL PNEUMONIA PNEUMONIA PNEUMONIA



In [79]: `import torch.optim as optim`

```
criterion = nn.CrossEntropyLoss(weight=class_weights,reduction='mean')
optimizer = optim.SGD(net.parameters(), lr=0.0001, momentum=0.9)
```

In []: `for epoch in range(50): # loop over the dataset multiple times`

```
    running_loss = 0.0
    for i, data in enumerate(train_loader, 0):
        # get the inputs; data is a list of [inputs, labels]
        inputs, labels = data

        # zero the parameter gradients
        optimizer.zero_grad()

        # forward + backward + optimize
        outputs = net(inputs)
        # print(outputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        # print statistics
        running_loss += loss.item()
        if i % 50 == 49: # print every 50 mini-batches
            print(f'[epoch + 1], {i + 1:5d}] loss: {running_loss / 200:.3f}')
            running_loss = 0.0

    print('Finished Training')
```

```
[1, 50] loss: 0.020
[1, 100] loss: 0.012
[1, 150] loss: 0.017
[2, 50] loss: 0.015
[2, 100] loss: 0.018
[2, 150] loss: 0.015
[3, 50] loss: 0.015
[3, 100] loss: 0.018
```


[3, 150] loss: 0.014
[4, 50] loss: 0.018
[4, 100] loss: 0.015
[4, 150] loss: 0.015
[5, 50] loss: 0.012
[5, 100] loss: 0.016
[5, 150] loss: 0.018
[6, 50] loss: 0.014
[6, 100] loss: 0.015
[6, 150] loss: 0.016
[7, 50] loss: 0.017
[7, 100] loss: 0.013
[7, 150] loss: 0.018
[8, 50] loss: 0.016
[8, 100] loss: 0.015
[8, 150] loss: 0.015
[9, 50] loss: 0.013
[9, 100] loss: 0.017
[9, 150] loss: 0.015
[10, 50] loss: 0.013
[10, 100] loss: 0.015
[10, 150] loss: 0.018
[11, 50] loss: 0.014
[11, 100] loss: 0.015
[11, 150] loss: 0.016
[12, 50] loss: 0.017
[12, 100] loss: 0.015
[12, 150] loss: 0.014
[13, 50] loss: 0.014
[13, 100] loss: 0.012
[13, 150] loss: 0.017
[14, 50] loss: 0.016
[14, 100] loss: 0.014
[14, 150] loss: 0.015
[15, 50] loss: 0.015
[15, 100] loss: 0.014
[15, 150] loss: 0.015
[16, 50] loss: 0.016
[16, 100] loss: 0.015
[16, 150] loss: 0.014
[17, 50] loss: 0.013
[17, 100] loss: 0.014
[17, 150] loss: 0.016
[18, 50] loss: 0.017
[18, 100] loss: 0.016
[18, 150] loss: 0.012
[19, 50] loss: 0.013
[19, 100] loss: 0.015
[19, 150] loss: 0.015
[20, 50] loss: 0.016
[20, 150] loss: 0.011
[21, 50] loss: 0.014
[21, 100] loss: 0.014
[21, 150] loss: 0.014
[22, 50] loss: 0.015
[22, 100] loss: 0.015
[22, 150] loss: 0.012
[23, 50] loss: 0.013
[23, 100] loss: 0.015
[23, 150] loss: 0.014
[24, 50] loss: 0.012
[24, 100] loss: 0.015

```

[24, 150] loss: 0.013
[25, 50] loss: 0.014
[25, 100] loss: 0.014
[25, 150] loss: 0.013
[26, 50] loss: 0.013
[26, 100] loss: 0.014
[26, 150] loss: 0.015
[27, 50] loss: 0.013
[27, 100] loss: 0.014
[27, 150] loss: 0.013
[28, 50] loss: 0.011
[28, 100] loss: 0.013
[28, 150] loss: 0.016
[29, 50] loss: 0.011
[29, 100] loss: 0.015
[29, 150] loss: 0.015
[30, 50] loss: 0.013
[30, 100] loss: 0.015

```

```

In [1]: # save
PATH = './xray_net.pth'
torch.save(net.state_dict(), PATH)
# reload
net = Net()
net.load_state_dict(torch.load(PATH))

```

```

-----
NameError                                Traceback (most recent call last)
Cell In[1], line 3
      1 # save
      2 PATH = './xray_net.pth'
----> 3 torch.save(net.state_dict(), PATH)
      4 # reload
      5 net = Net()

NameError: name 'torch' is not defined

```

```

In [ ]: xray_testdataset = datasets.ImageFolder(root='C:/Users/DellDepot/Downloads/che
                                                transform=data_transform)
test_loader = torch.utils.data.DataLoader(xray_testdataset,
                                           batch_size=32, shuffle=True,
                                           num_workers=4)

```

```

In [ ]: dataiter = iter(test_loader)
images, labels = next(dataiter)

# print images
imshow(torchvision.utils.make_grid(images))
print('GroundTruth: ', ' '.join(f'{classes[labels[j]]:5s}' for j in range(32)))

```

```

In [ ]: outputs = net(images)

```

```

In [ ]: _, predicted = torch.max(outputs, 1)

print('Predicted: ', ' '.join(f'{classes[predicted[j]]:5s}'
                              for j in range(32)))

```

```

In [ ]: correct = 0
total = 0

```

```

with torch.no_grad():
    for data in test_loader:
        images, labels = data
        outputs = net(images)
        _, predicted = torch.max(outputs.data, 1)
        #print(predicted, labels)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

print(f'Accuracy of the network on the 10000 test images: {100 * correct // to
print(total)
print(labels)

```

```

In [ ]: correct_pred = {classname: 0 for classname in classes}
total_pred = {classname: 0 for classname in classes}

with torch.no_grad():
    for data in test_loader:
        images, labels = data
        outputs = net(images)
        _, predictions = torch.max(outputs, 1)
        for label, prediction in zip(labels, predictions):
            if label == prediction:
                correct_pred[classes[label]] += 1
                total_pred[classes[label]] += 1

for classname, correct_count in correct_pred.items():
    accuracy = 100 * float(correct_count) / total_pred[classname]
    print(f'Accuracy for class: {classname:5s} is {accuracy:.1f} %')

```

```

In [ ]: could not run here, i used google

```

```

In [ ]: import torch
import torch.nn as nn
import torch.optim as optim

# Define your model architecture
class MyModel(nn.Module):
    def __init__(self):
        super(MyModel, self).__init__()
        self.fc1 = nn.Linear(100, 64)
        self.fc2 = nn.Linear(64, 1)
        self.relu = nn.ReLU()
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        x = self.relu(self.fc1(x))
        x = self.sigmoid(self.fc2(x))
        return x

model = MyModel()

# Specify the Adam optimizer
adam = optim.Adam(model.parameters(), lr=0.001)

# Define the loss function
criterion = nn.BCELoss()

# Train your model
for epoch in range(num_epochs):

```

```

# Forward pass
outputs = model(inputs)
loss = criterion(outputs, labels)

# Backward and optimize
optimizer.zero_grad()
loss.backward()
optimizer.step()

```

```

In [78]: import torch
from torchvision import transforms, datasets

data_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5, 0.5, 0.5],
                          std=[0.5, 0.5, 0.5]),
    transforms.Resize([100,100])
])
xray_dataset = datasets.ImageFolder(root='C:/Users/DellDepot/Downloads/chest_x
                                     transform=data_transform)
train_loader = torch.utils.data.DataLoader(xray_dataset,
                                             batch_size=32, shuffle=True,
                                             num_workers=0)

```

```

In [82]: import torch.nn as nn
import torch.nn.functional as F

class Net(nn.Module):
    def __init__(self):
        super().__init__()

        # Notes: What does nn.Conv2d do?
        # nn.Conv2d(in channel, out channel, kernel_size)
        # in_channel = 3 because of RGB
        # out_channel = 6 features
        # kernel_size = size of kernel (acts like a feature detector throughout)
        self.conv1 = nn.Conv2d(3, 6, 5)
        # converts image 3 * 100 * 100 to image 6 * 96 * 96
        self.pool = nn.MaxPool2d(2, 2)
        # converts image 6 * 96 * 96 to image 6 * 48 * 48
        self.conv2 = nn.Conv2d(6, 16, 5)
        # converts image 6 * 48 * 48 to image 16 * 44 * 44
        self.fc1 = nn.Linear(16 * 22 * 22, 44 * 44) #
        self.fc2 = nn.Linear(44 * 44, 20 * 20)
        self.fc3 = nn.Linear(20 * 20, 2) #400 by 2

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = torch.flatten(x, 1)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x

net = Net()

```

```
In [83]: import torch.optim as optim

import torch
import torch.nn as nn
import torch.optim as optim

criterion = nn.CrossEntropyLoss(weight=class_weights, reduction='mean')
#optimizer = optim.SGD(net.parameters(), lr=0.0001, momentum=0.9)

# Specify the Adam optimizer
optimizer = optim.Adam(net.parameters(), lr=0.0001)#adam = optim.Adam(model.pa

# Define the loss function
#criterion = nn.BCELoss()
```

```
In [94]: for epoch in range(10): # loop over the dataset multiple times

    running_loss = 0.0
    for i, data in enumerate(train_loader, 0):
        # get the inputs; data is a list of [inputs, labels]
        inputs, labels = data

        # zero the parameter gradients
        optimizer.zero_grad()

        # forward + backward + optimize
        outputs = net(inputs)
        # print(outputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        # print statistics
        running_loss += loss.item()
        if i % 50 == 49: # print every 50 mini-batches
            print(f'[{epoch + 1}, {i + 1:5d}] loss: {running_loss / 200:.3f}')
            running_loss = 0.0

    print('Finished Training')
```

```
[1,   50] loss: 0.023
[1,  100] loss: 0.023
[1,  150] loss: 0.024
[2,   50] loss: 0.023
[2,  100] loss: 0.026
[2,  150] loss: 0.027
[3,   50] loss: 0.024
[3,  100] loss: 0.023
[3,  150] loss: 0.027
[4,   50] loss: 0.023
[4,  100] loss: 0.024
[4,  150] loss: 0.026
[5,   50] loss: 0.023
[5,  100] loss: 0.023
[5,  150] loss: 0.026
[6,   50] loss: 0.023
[6,  100] loss: 0.022
[6,  150] loss: 0.026
[7,   50] loss: 0.028
[7,  100] loss: 0.025
[7,  150] loss: 0.022
```



```

    for data in test_loader:
        images, labels = data
        outputs = net(images)
        _, predicted = torch.max(outputs.data, 1)
        #print(predicted, labels)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

print(f'Accuracy of the network on the 624 test images: {100 * correct // total} %')
print(total)
print(labels)

```

Accuracy of the network on the 624 test images: 77 %
624
tensor([0, 0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 0, 1])

In [102]...

```

classes = ['NORMAL', 'PNEUMONIA']

correct_pred = {classname: 0 for classname in classes}
total_pred = {classname: 0 for classname in classes}

with torch.no_grad():
    for data in test_loader:
        images, labels = data
        outputs = net(images)
        _, predictions = torch.max(outputs, 1)
        for label, prediction in zip(labels, predictions):
            if label == prediction:
                correct_pred[classes[label]] += 1
                total_pred[classes[label]] += 1

for classname, correct_count in correct_pred.items():
    accuracy = 100 * float(correct_count) / total_pred[classname]
    print(f'Accuracy for class: {classname:5s} is {accuracy:.1f} %')

```

Accuracy for class: NORMAL is 40.2 %
Accuracy for class: PNEUMONIA is 99.2 %

Epoch 3 with Adam Accuracy for class: NORMAL is 40.2 % Accuracy for class: PNEUMONIA is 99.2 %

In []: